

Hex Terrains Framework

Hex Terrain Framework – Modular, High-Performance Hex-Based World System for Unity DOTS

Documentation

Instruction (<https://www.freewebtime.com/hexterrains/getting-started.html>)

Class Reference (<https://www.freewebtime.com/hexterrains/api/Fwt.HexTerrains.html>)



The Hex Terrain Framework is a fully modular, runtime-editable, high-performance hexagonal terrain system built for Unity's **ECS (DOTS)** architecture. Designed from the ground up for large-scale simulation and strategy games, it offers a Photoshop-like editing workflow with full brush-based control, multichannel data layering, and dynamic surface generation — all with performance and flexibility in mind.

Whether you're building a game like *Civilization*, *Factorio*, or your own simulation-heavy world, this framework provides everything you need to create, edit, render, and interact with vast hex-based environments.

Core Features

- **Brush-Based Terrain Editing** Edit terrain in real time with brush tools that support size, opacity, stamping, alt-sampling (like a color picker), and auto-painting — just like Photoshop.
- **Dynamic Terrain Creation** Create new terrains at runtime with custom dimensions, cell resolution, and layer configuration. Resize or regenerate terrain without restarting.
- **Modular Data Layer System** Add or remove data layers (heightmap, biomes, countries, items, entities, etc.) on demand. Each layer is self-contained and supports dirty flagging and job-safe dependency tracking.
- **Surface Layers & Water Support** Render complex layered surfaces (e.g., ground, water, snow, oil) using custom or built-in mesh generators. Water is its own elevation map — fully interactive and editable.
- **Highly Optimized Mesh Generation** Meshes are generated using Burst-compiled jobs with support for simplified or detailed topology. You can extend it with your own generators for custom visuals.
- **Minimap with Viewport Sync** Real-time minimap renders any data layer texture. Click-to-center camera control. Camera position indicator with normalized tracking included.
- **HDRP & URP Ready** Sample content provided for both pipelines. Choose your look and customize with Shader Graph.
- **Custom UI Toolkit-Based Editor** In-game terrain editor built with Unity UI Toolkit, featuring fully data-bound property windows, tool panels, and a modular view model system. Easily extend with your own tools.
- **Save/Load Support** Modular binary serialization built-in. Save and load entire terrain states, including all data layers, runtime changes, and surface configuration.
- **Wraparound Map Support** Connect terrain edges horizontally and vertically (e.g., like a globe or Civ-style map). Seamless camera movement across terrain boundaries.

Technical Highlights

- Built entirely on **Unity DOTS (ECS 1.0)**
- **Burst-accelerated hex math** with axial/offset and flat/point top options
- Minimal allocations and full job safety — optimized for large terrains
- Data layers store **per-chunk dirty flags** and **read/write job handles**
- Render system designed for **Graphics.DrawMeshInstanced**, not hybrid entities
- Designed for **multi-terrain projects** (e.g., Earth, Moon, Mars...)

Ideal For

- 4X and Strategy Games
- Simulation and Economy Games
- City Builders
- Custom World Editors
- ECS Learning Projects
- Games that require **fully interactive hex terrain with high performance**

Included

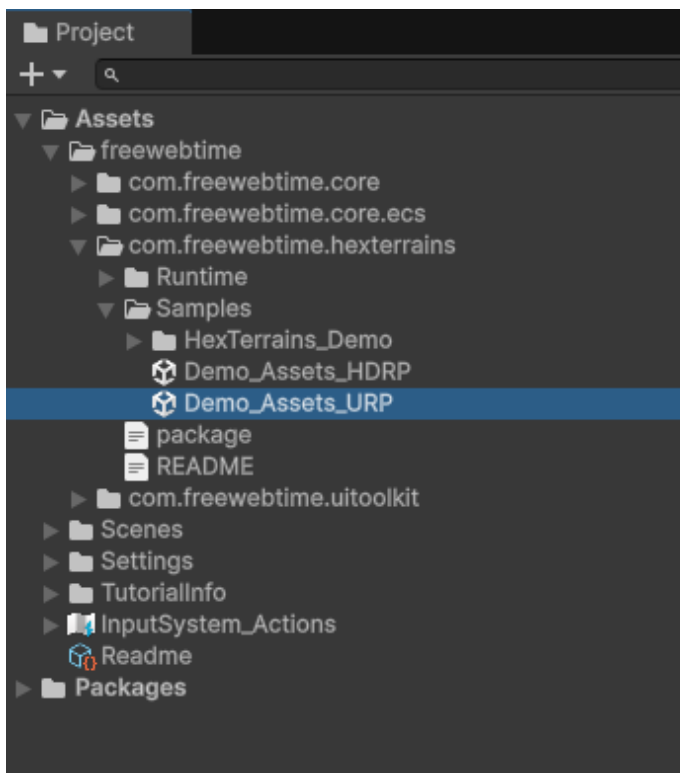
- Full source code (C# + DOTS)
- Demo project with HDRP and URP samples
- In-game terrain editor with sample tools
- Procedural brushes and auto-painting systems
- Modular, extendable UI framework

Getting Started



Samples Demo Scene

1. Install package from asset store.
2. Now you can either copy contents into your "ProjectRoot/Packages" folder (to make it an embedded packages) or you can use it from the Assets folder.
3. If you've decided to use it from the Assets folder
 - Open folder "freewebtime/com.freewebtime.hexterrains/Samples"
 - Extract one of two .unitypackage archives into your project depending on your render pipeline: Demo_Assets_URP (if you use Universal Render Pipeline) or Demo_Assets_HDRP (if you use HDRP render pipeline).

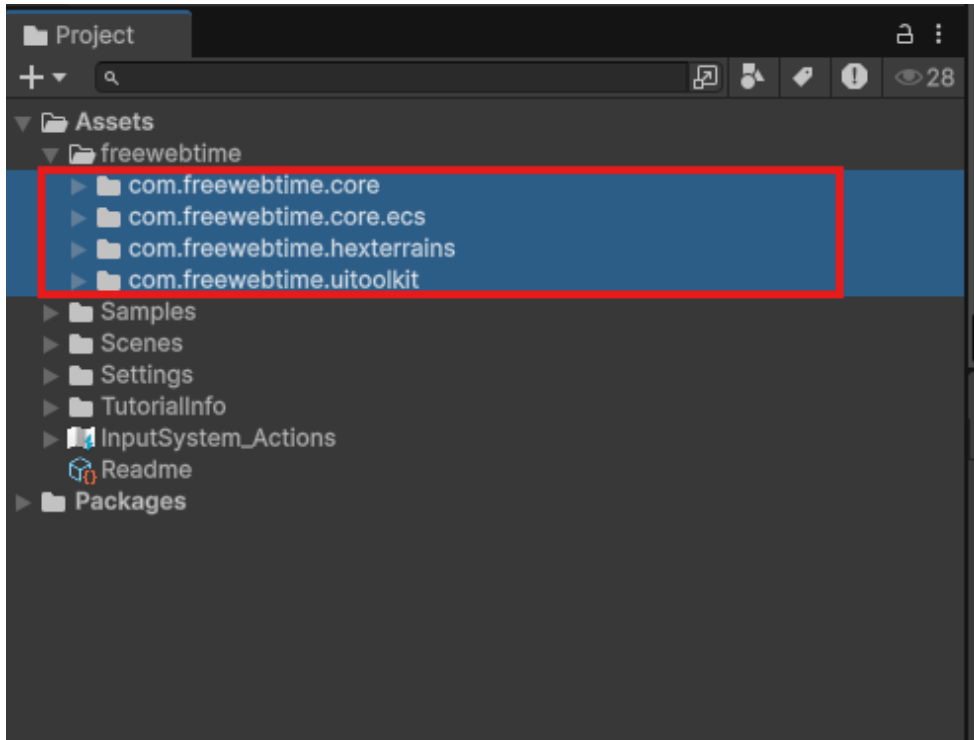


4. If you've decided to copy package contents into your Packages folder

- Copy those four folders into your "ProjectRoot/Packages" folder

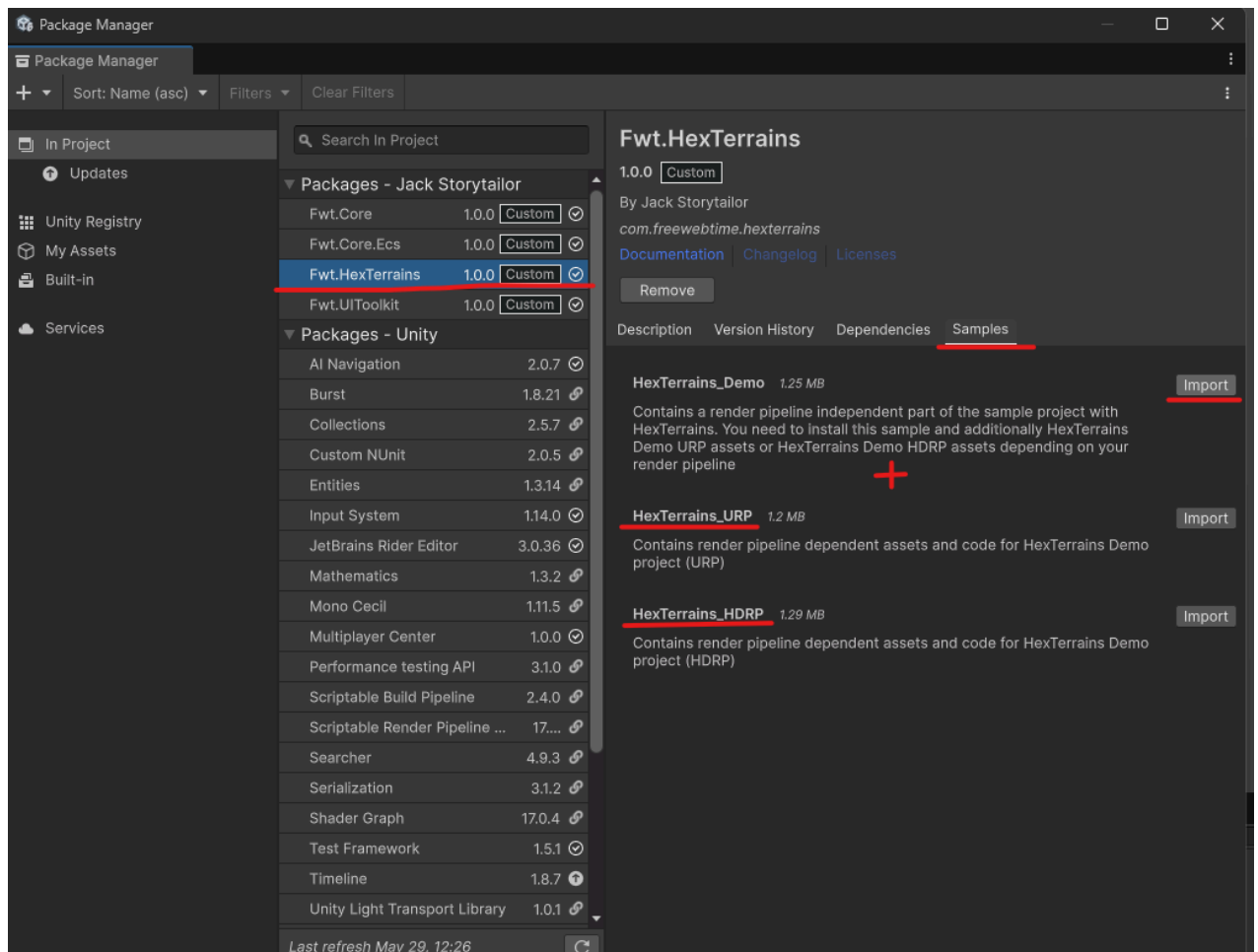
Assets/freewebtime/com.freewebtime.core Assets/freewebtime/com.freewebtime.core.ecs

Assets/freewebtime/com.freewebtime.hexterrains Assets/freewebtime/com.freewebtime.uitoolkit

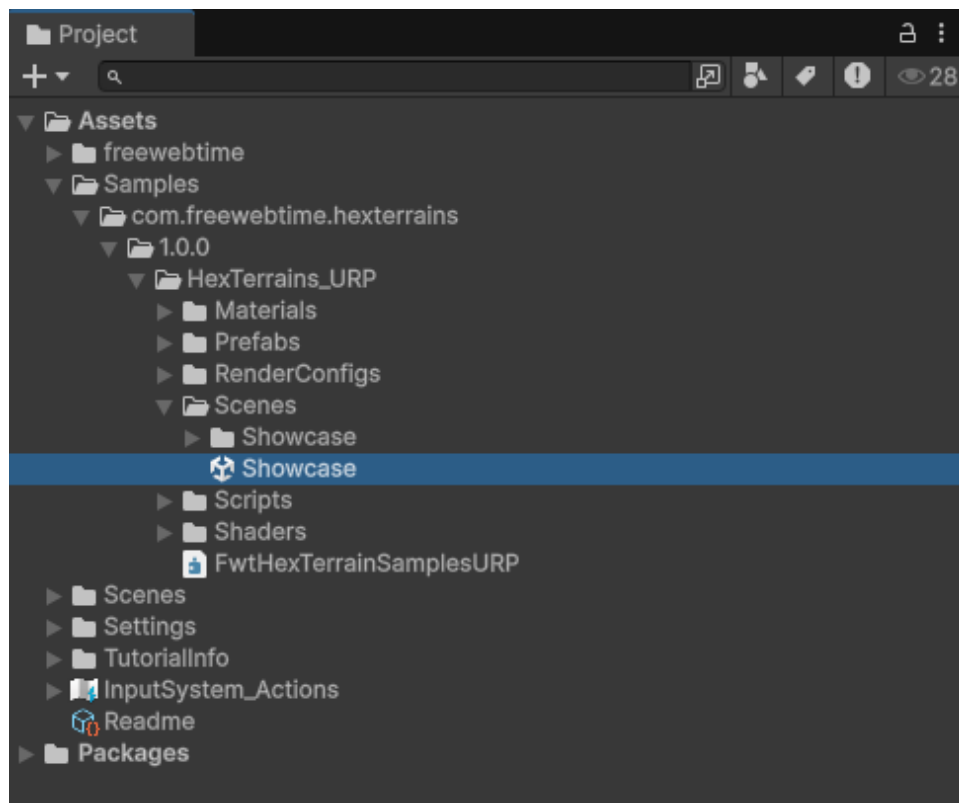


- Open Package Manager window (Window -> Package Manager)
- Select the Fwt.HexTerrains package
- In right section of the window find tabs "Description", "Version History", "Dependencies", "Samples". Select "Samples"

- Import HexTerrains Demo core sample
- Additionally import one of two following samples depending on your render pipeline: "HexTerrains Demo URP assets" or "HexTerrains Demo HDRP assets"



5. After sample projects are imported into your project, navigate to folder "Assets/Samples/com.freewebtime.hexterrains/1.0.0/HexTerrains_URP" or "Assets/Samples/com.freewebtime.hexterrains/1.0.0/HexTerrains_HDRP" depending on your render pipeline.
6. From there navigate to "Scenes" folder and open Showcase scene

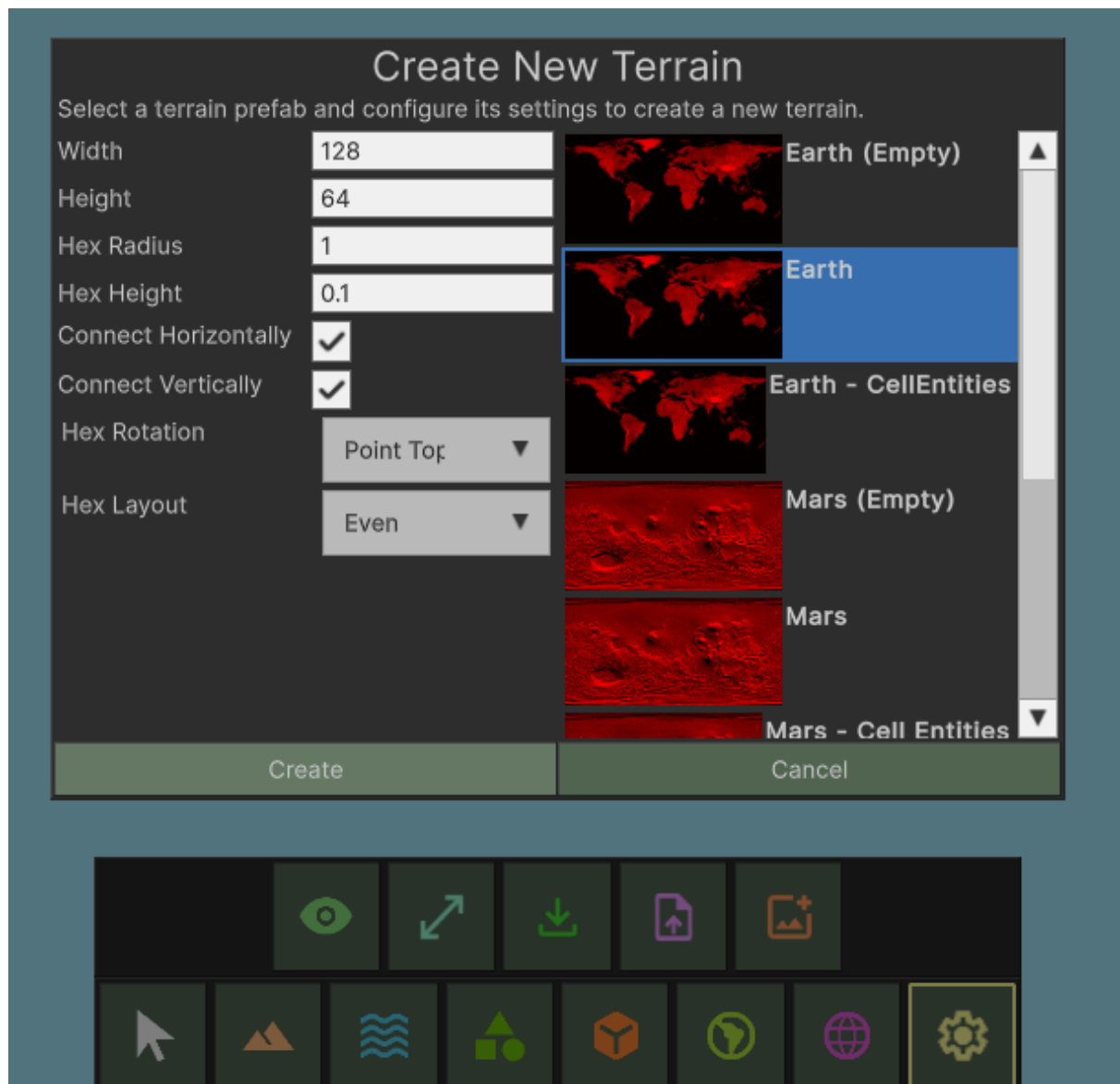


7. Press "Play" button

8. Select "System" User Tool category, then "Create New"



9. In opened window select a terrain prefab and put the settings you want.



10. Press "Create" button

11. Enjoy!



In-game terrain editor

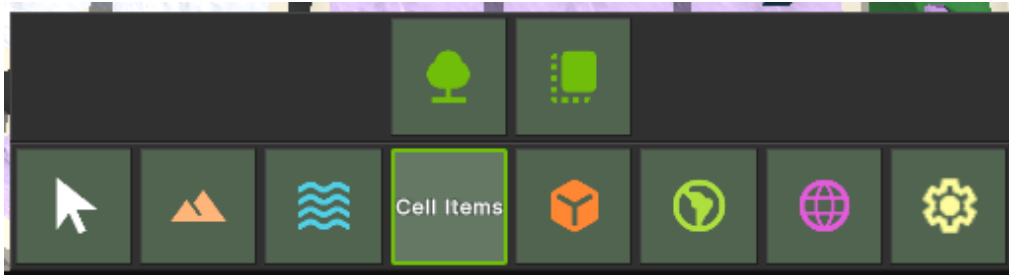
1. In-game editor is a State Machine called (see class `UserToolStateMachine`). State Machine has a set of User Tool States like Deform Terrain, Paint Biomes, Paint Cell Items, Paint Countries, etc. Every `UserToolState` communicates with a Terrain using a proxy implementation of the `ITexTerrainAPI` (see `HexTerrainAPI` class).
2. All User Tools are split between a set of categories:
 - Ground Tools (Deform, Level, Noise, Smooth, Heightmap, Paint Biome, Auto-paint Biome, Stamp Height, Stamp Biome)



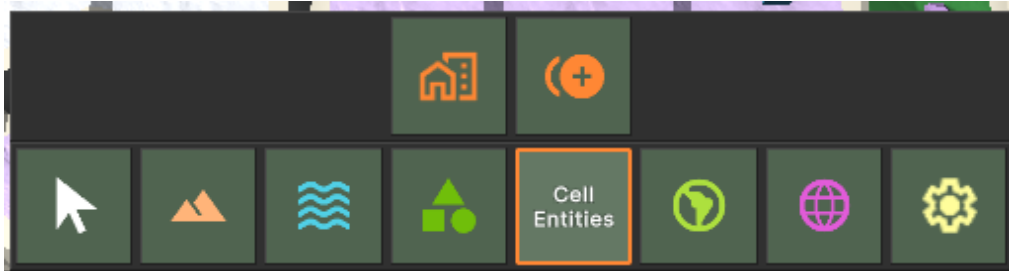
- Water Tools (Deform, Level, Noise, Smooth, Heightmap, Paint Biome, Auto-paint Biome, Stamp Height, Stamp Biome)



- Cell Items (Paint Cell Items, Stamp Cell Items)



- Cell Entities (Paint Cell Entities, Stamp Cell Entities)



- Provinces (Paint Provinces, Stamp Provinces)



- Countries (Paint Countries, Stamp Countries)



- System (Toggle Visibility, Resize Terrain, Save Terrain, Load Terrain, Create New)



View modes

Hex Terrain supports rendering in a different view modes. View mode is a number, the value of the `HexTerrainViewMode` component on a Terrain Entity.

```

/// <summary>
/// Contains the current view mode of the terrain.
/// </summary>
public struct HexTerrainViewMode : IComponentData
{
    /// <summary>
    /// The current view mode of the terrain.
    /// </summary>
    public VersionValue<byte> Value;
}

```

To change a view mode, click on a respective button above the minimap. There are four built-in view modes already, but it's very simple to expand the list:

- Normal View Mode (Regular terrain view)



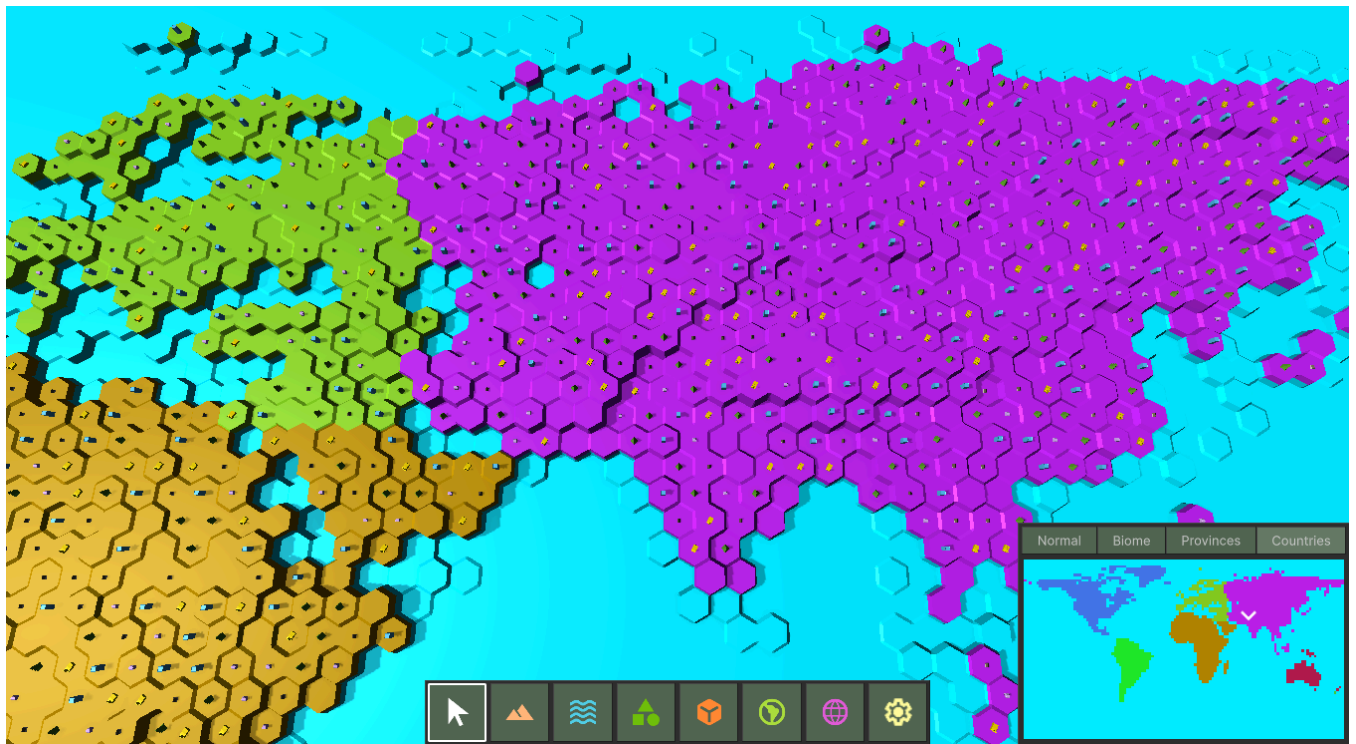
- Biomes View Mode (Paints each cell to a color of it's biome, see [ColorPalette](#) on [HexSurfaceLayerAuthoring](#) mono behaviour)



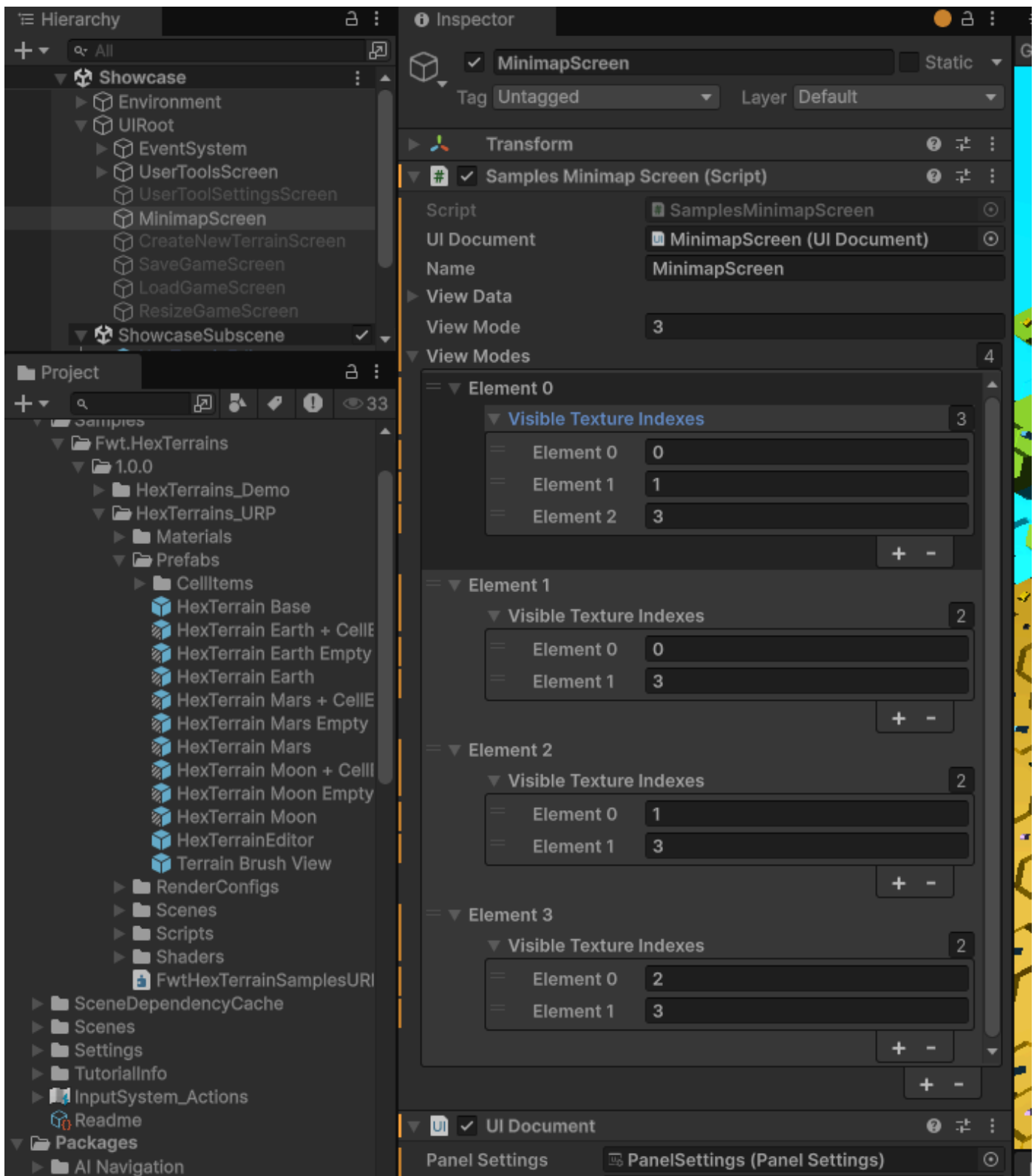
- Provinces View Mode



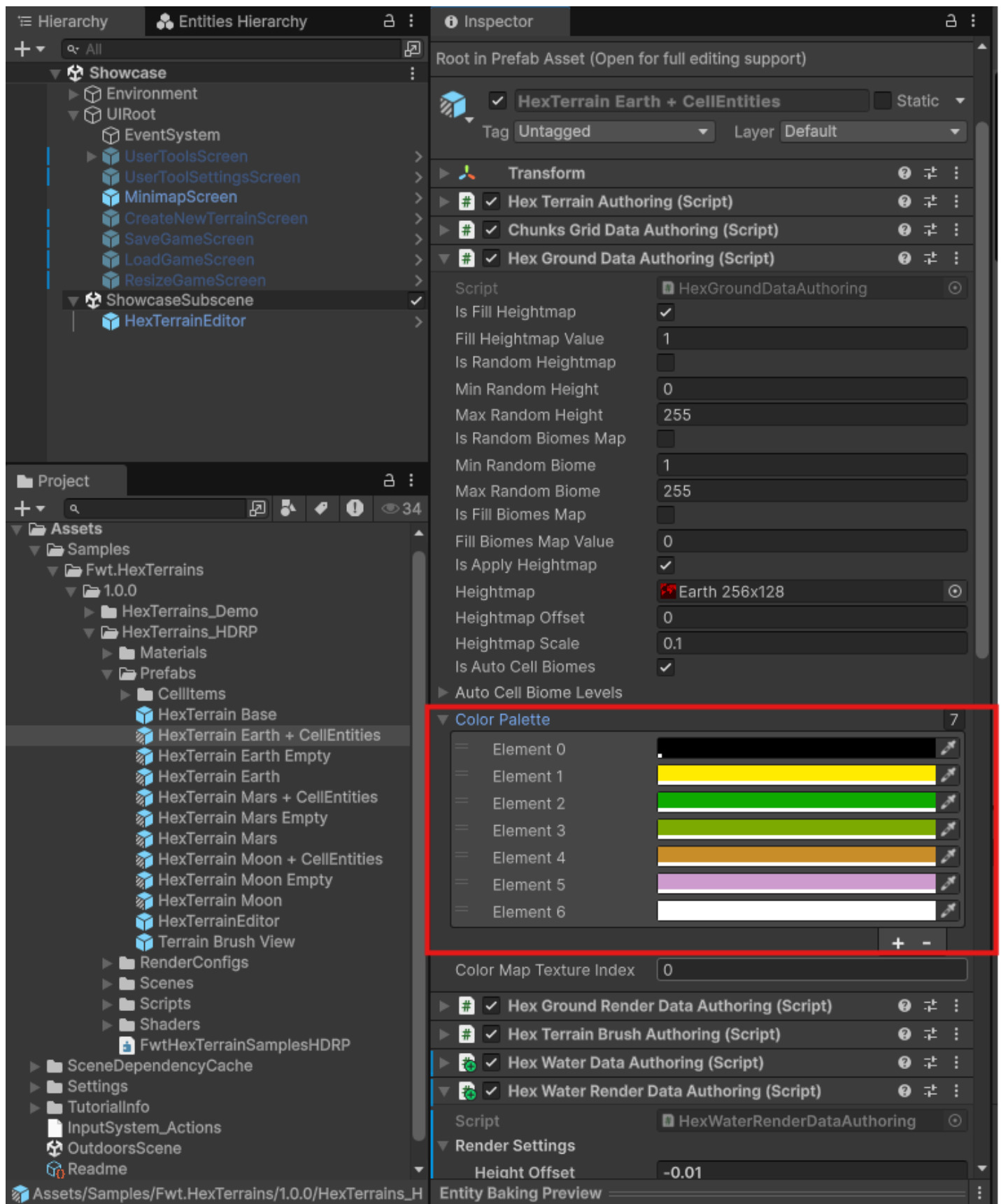
- Countries View Mode



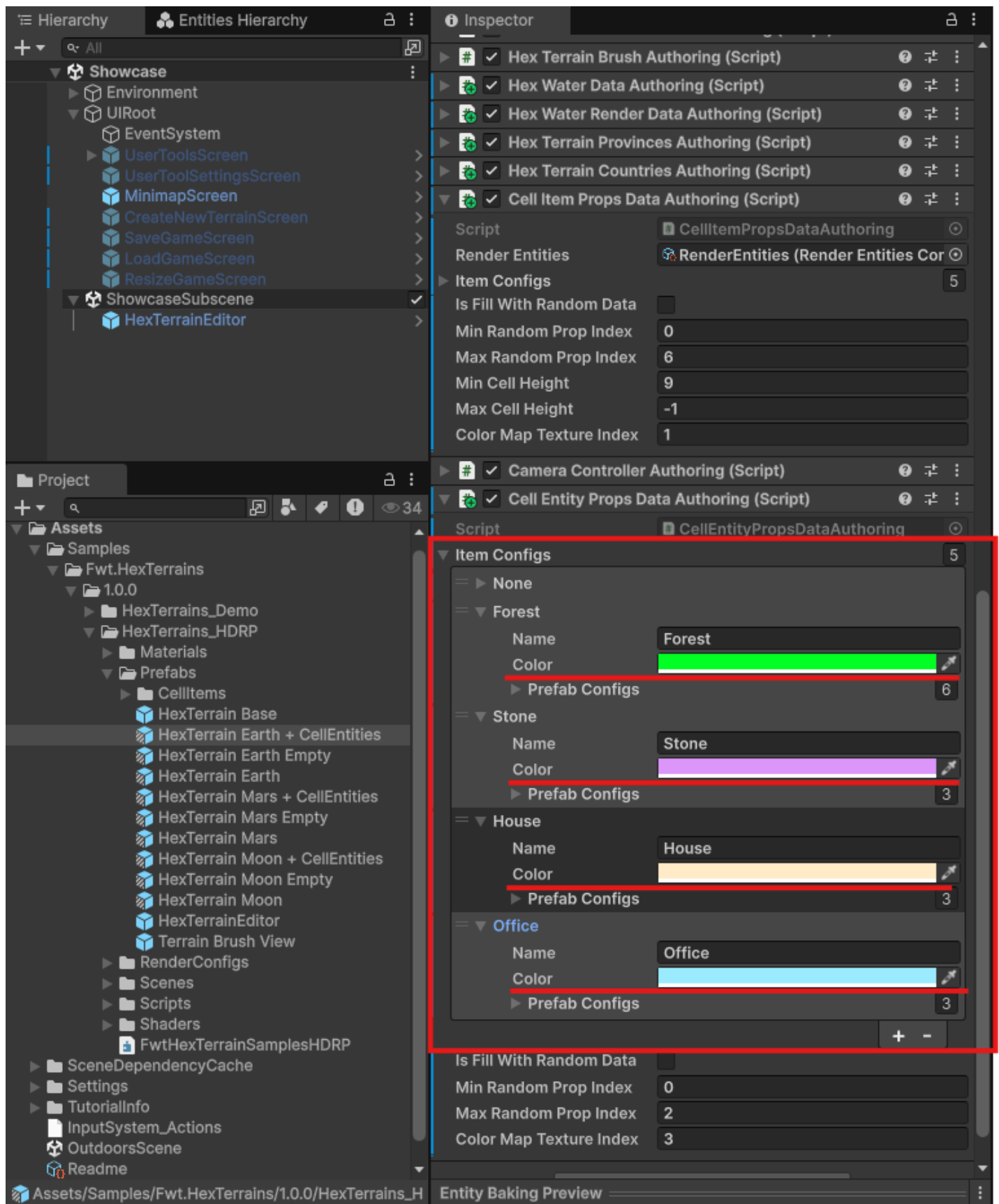
To setup a ViewMode, take a look at the `MinimapScreen` prefab, the `SamplesMinimapScreen` component, the `ViewModes` list. Here you can setup which color textures are painted on the Minimap when terrain is in the respective ViewMode.



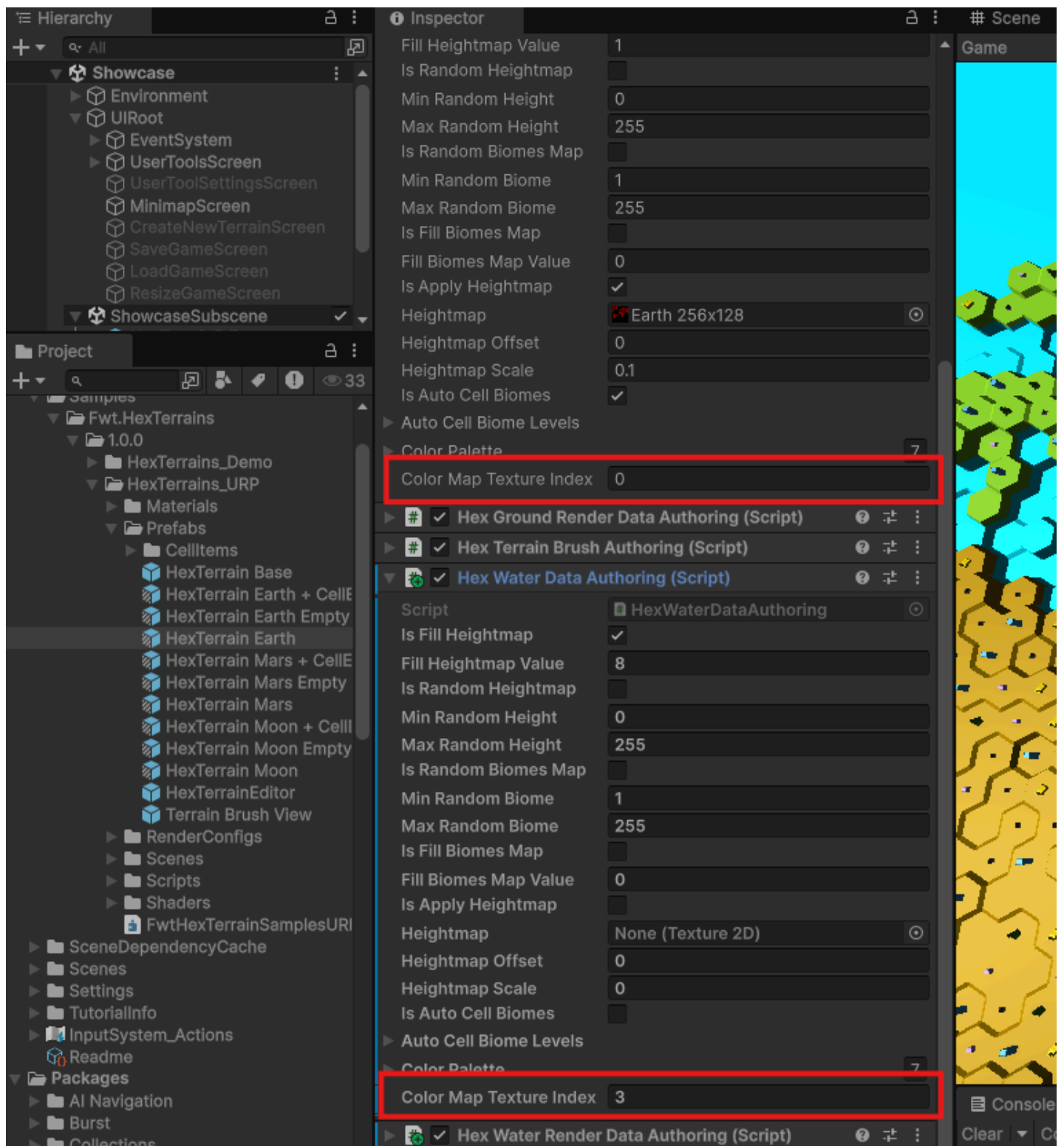
Color texture is a read/write enabled Texture2D that is filled with calculated colors from terrain. For instance, each biome has it's own color for minimap (see `ColorPalette` list on `SurfaceLayer` authoring component)

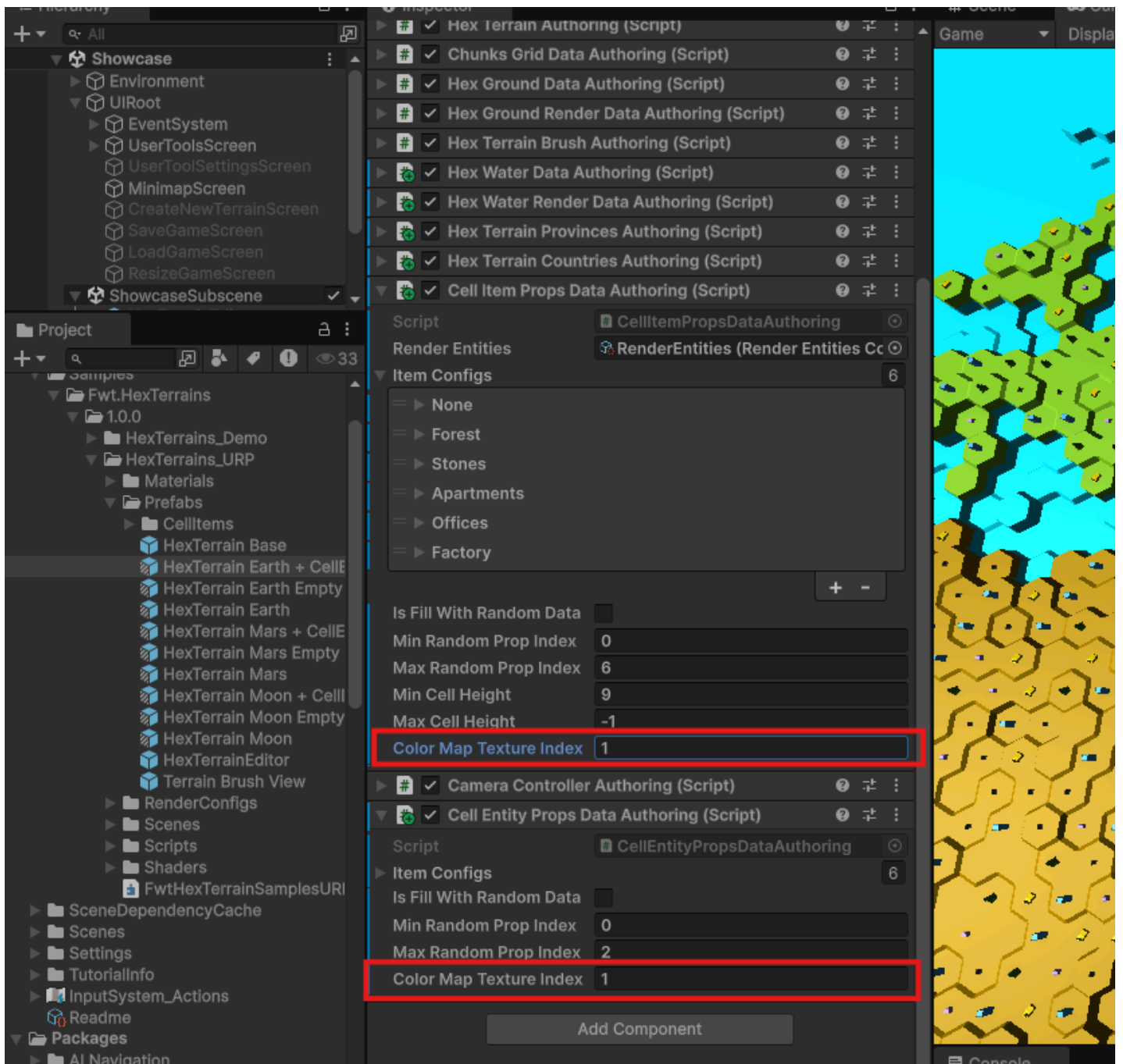


or, for example, the Color setup for each Cell Entity:

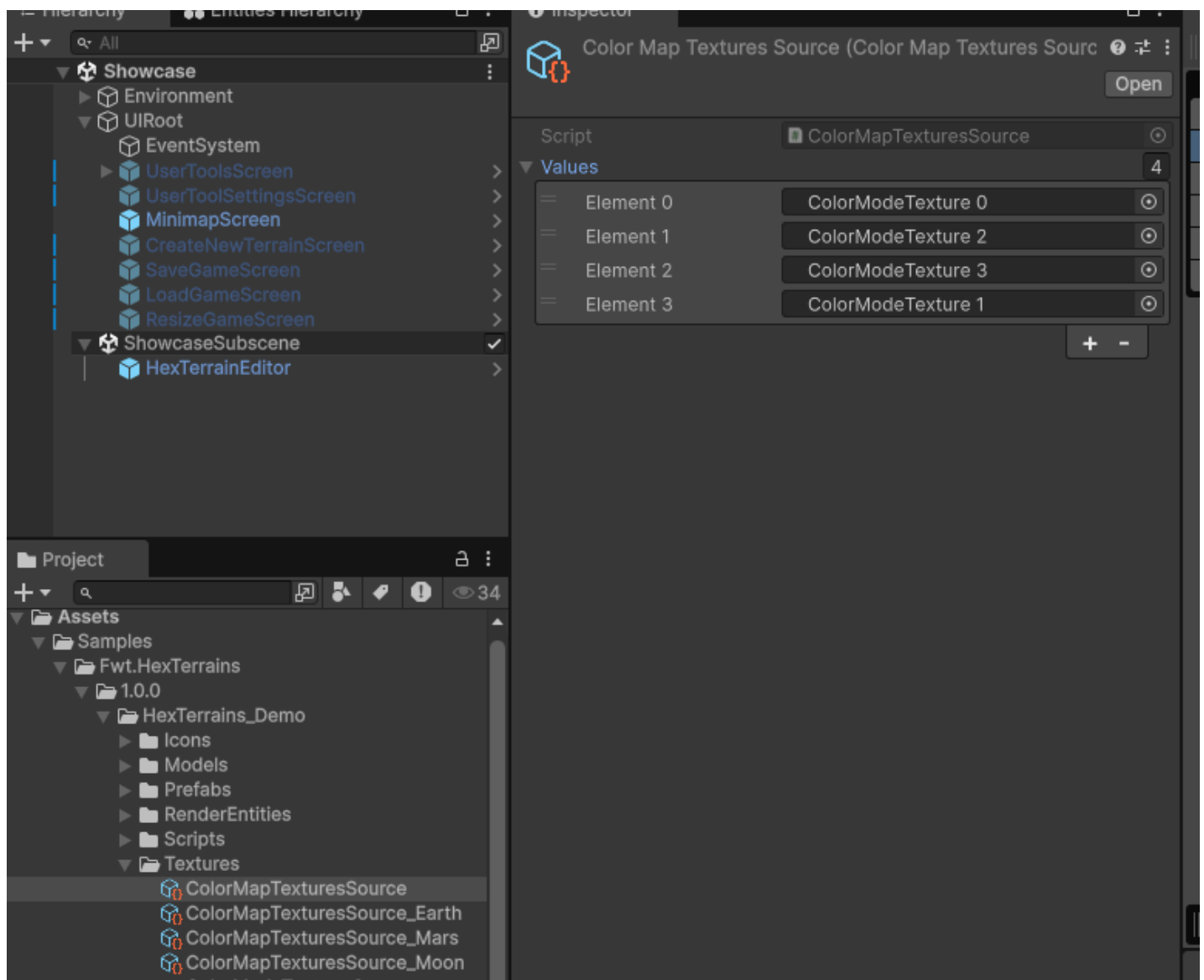


Now, take a look at the field `ColorMapTextureIndex` of each renderable terrain layer (Surface, ByteArea, CellItems, CellEntities). This value tells on which texture to copy colors from the layer's color palette.

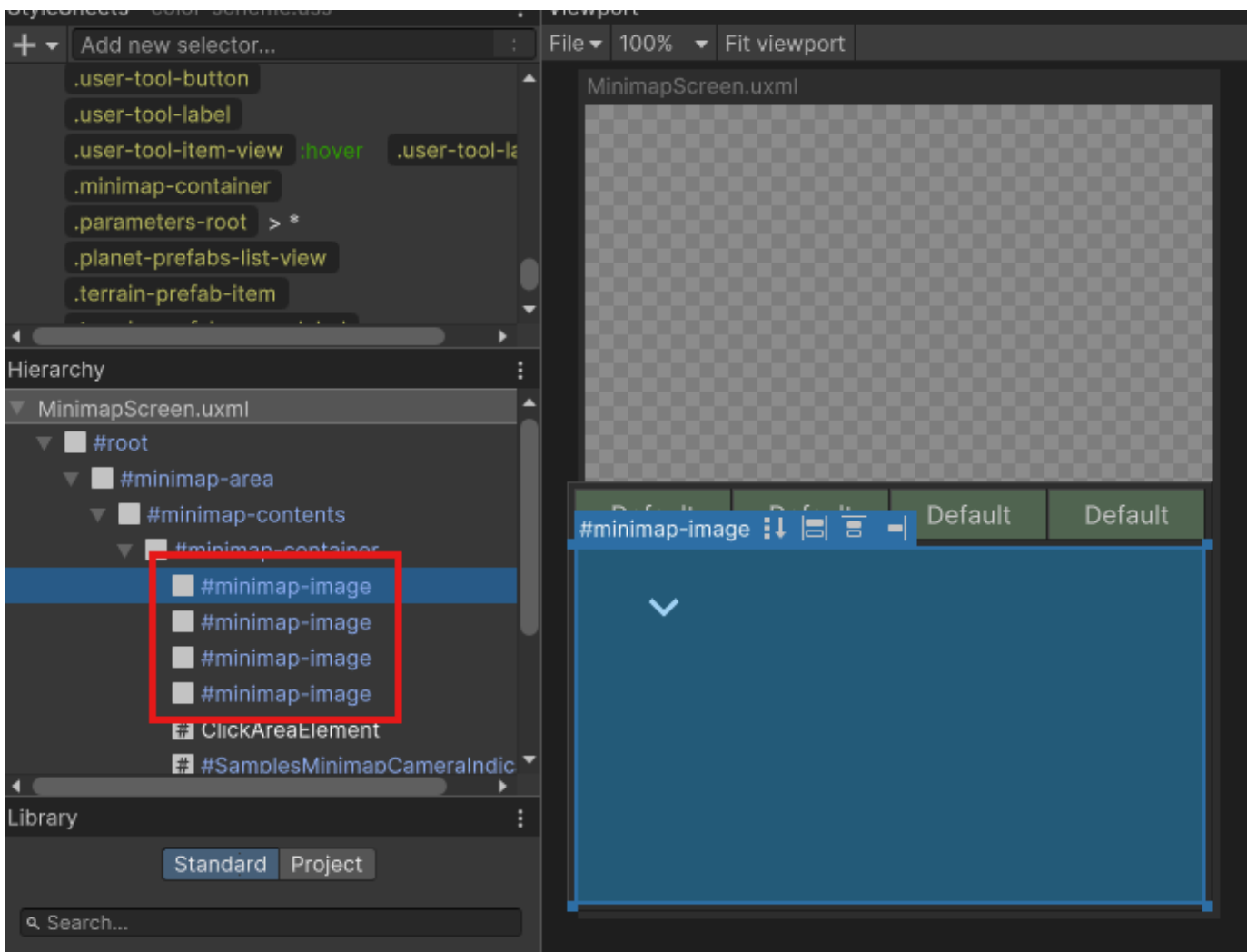




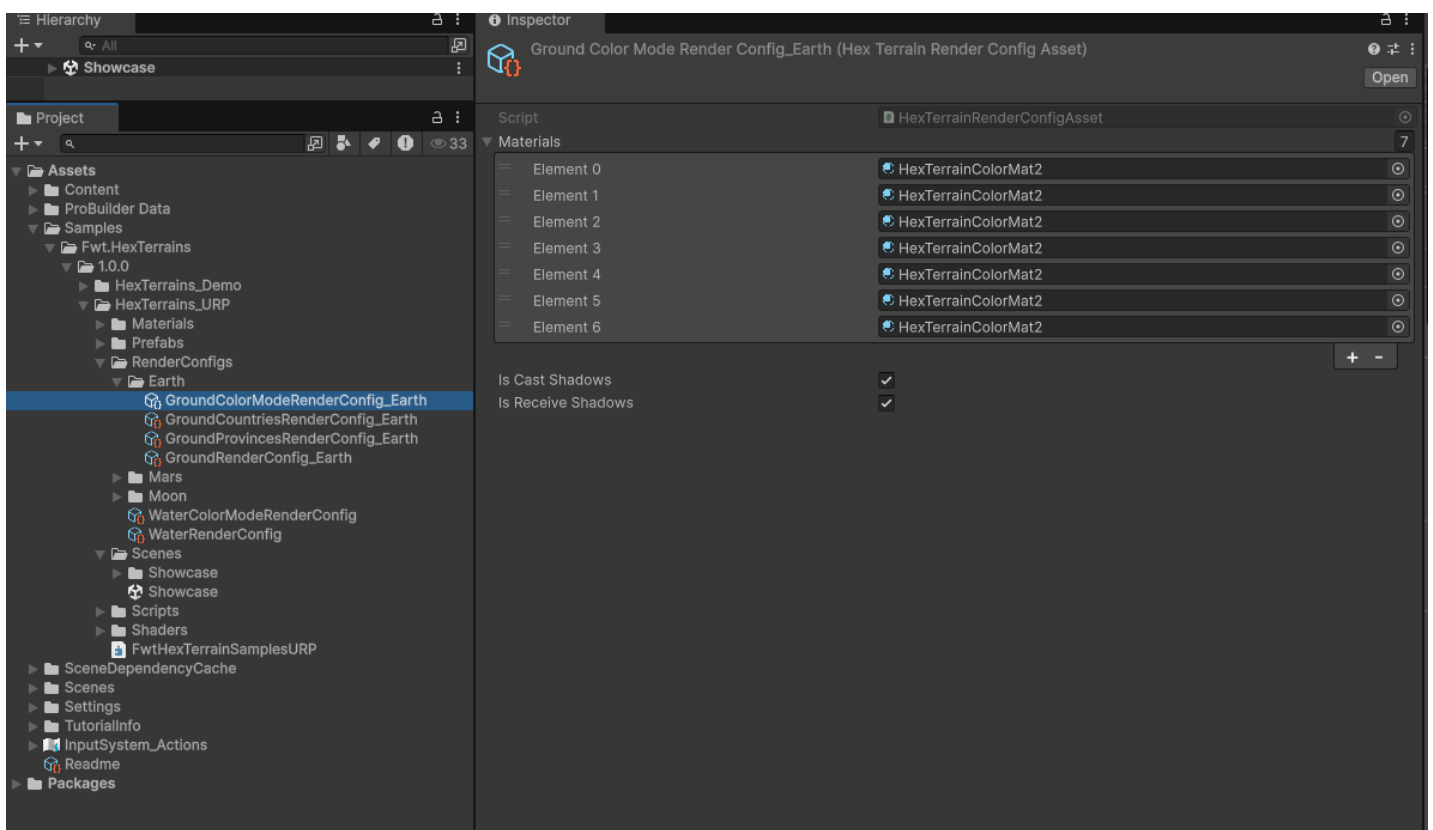
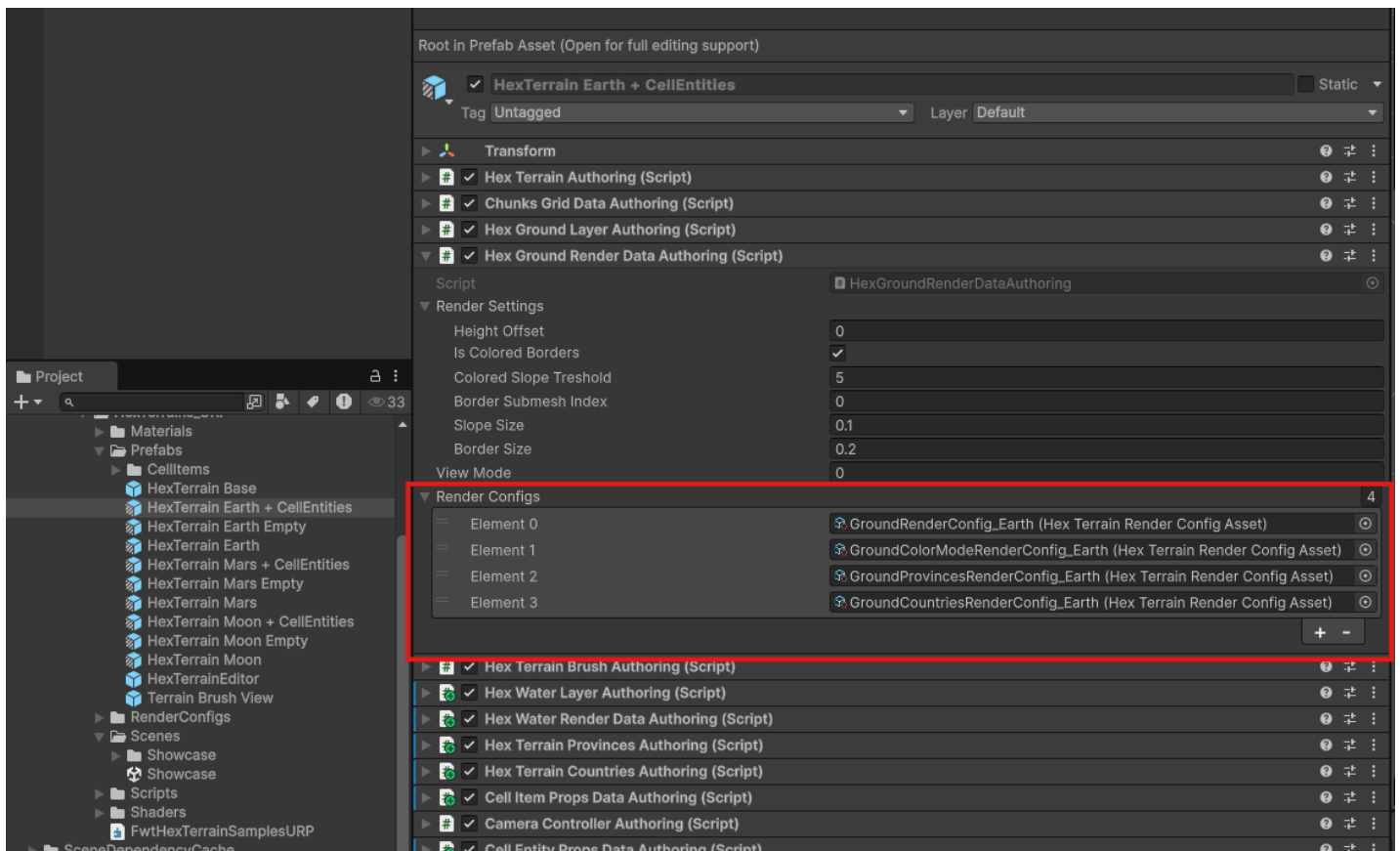
All textures are stored into the `ColorMapTexturesSource` scriptable object:

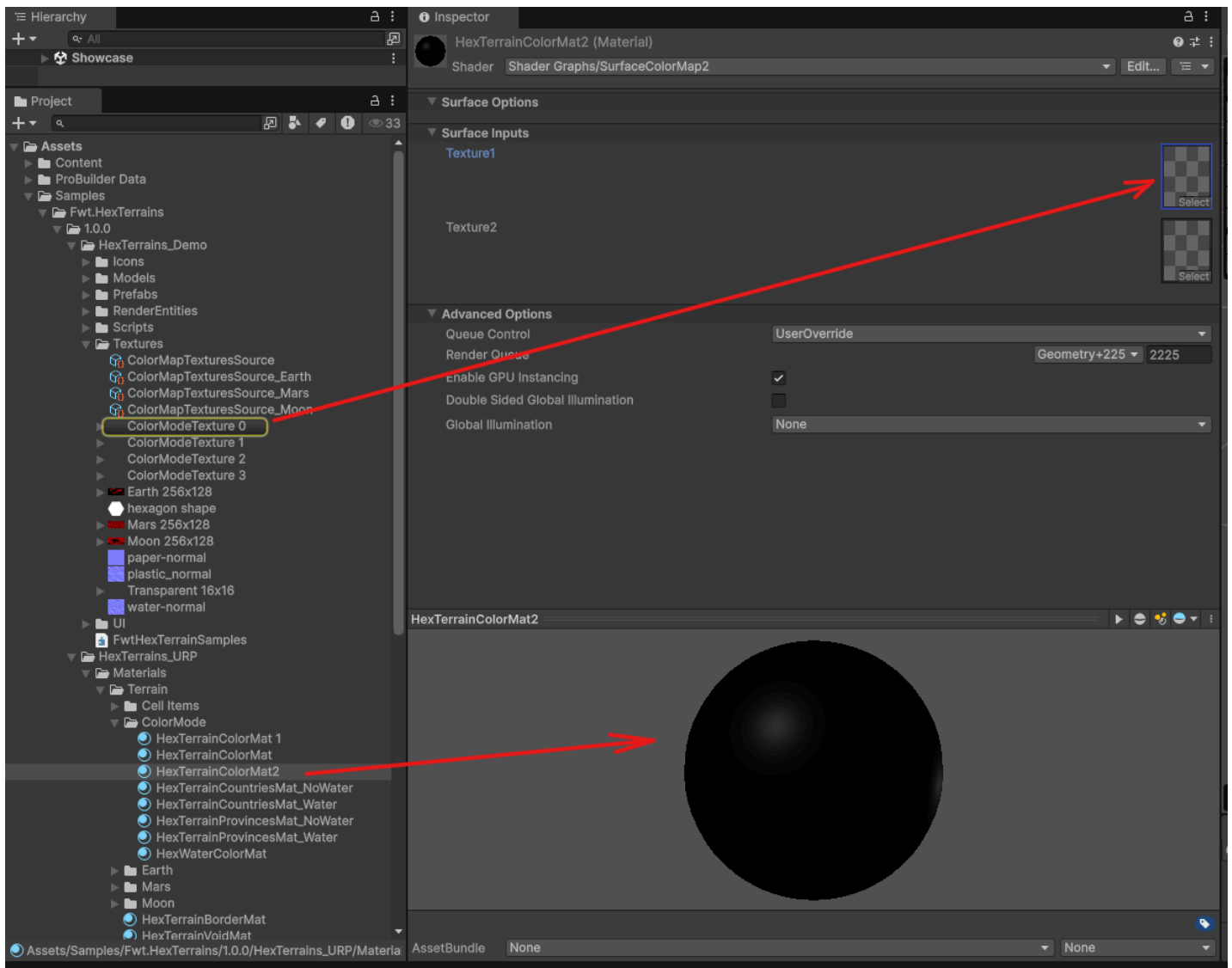


Then setup the UI, so it has room for all the ColorMap textures you have (currently it displays 4 textures)



And finally, when the terrain switches to a new View Mode, it starts rendering by different set of materials. If those materials use one of your Color Texture (where the terrain color data is copied), you will see the terrain painted in colors of the color map.

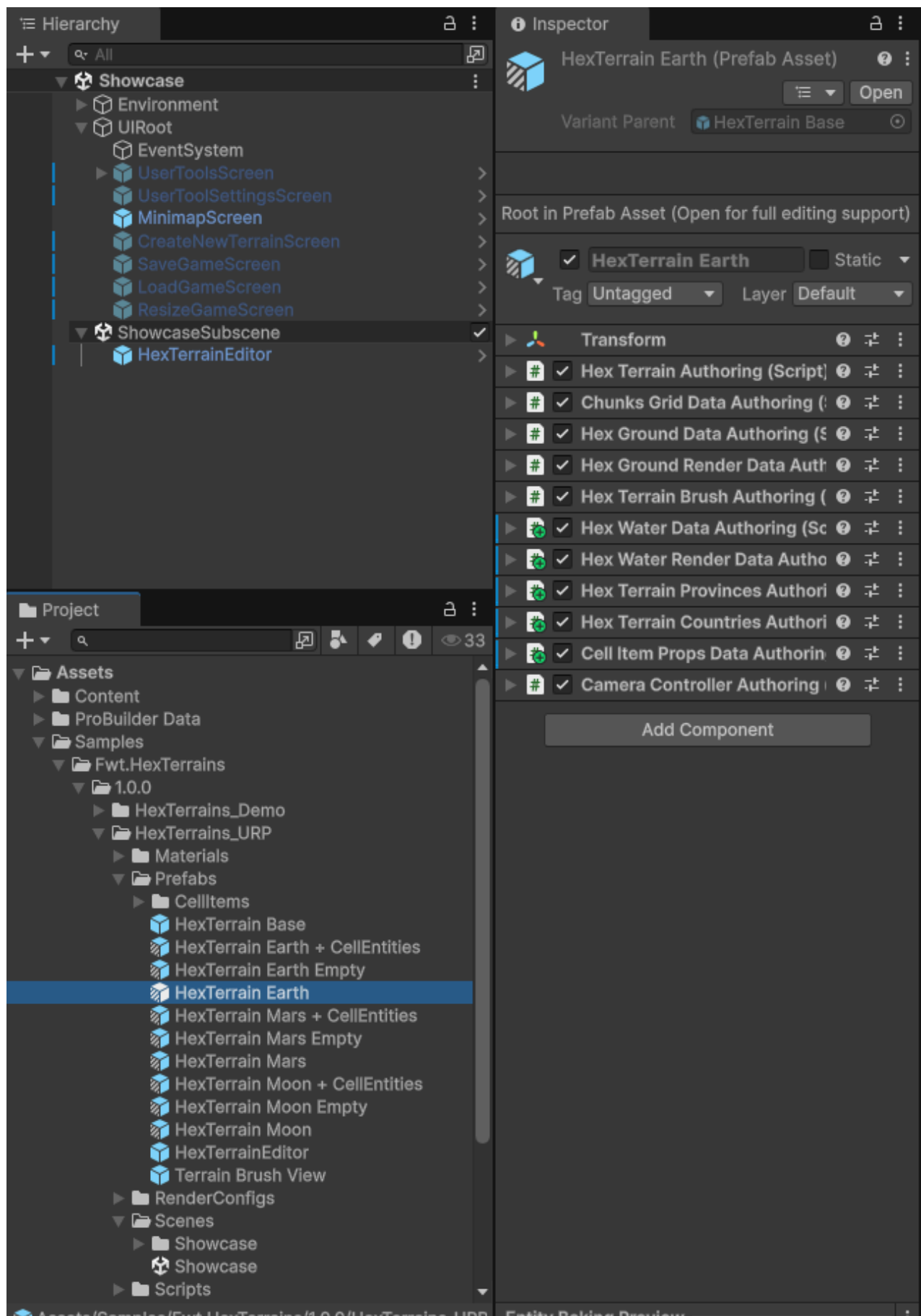




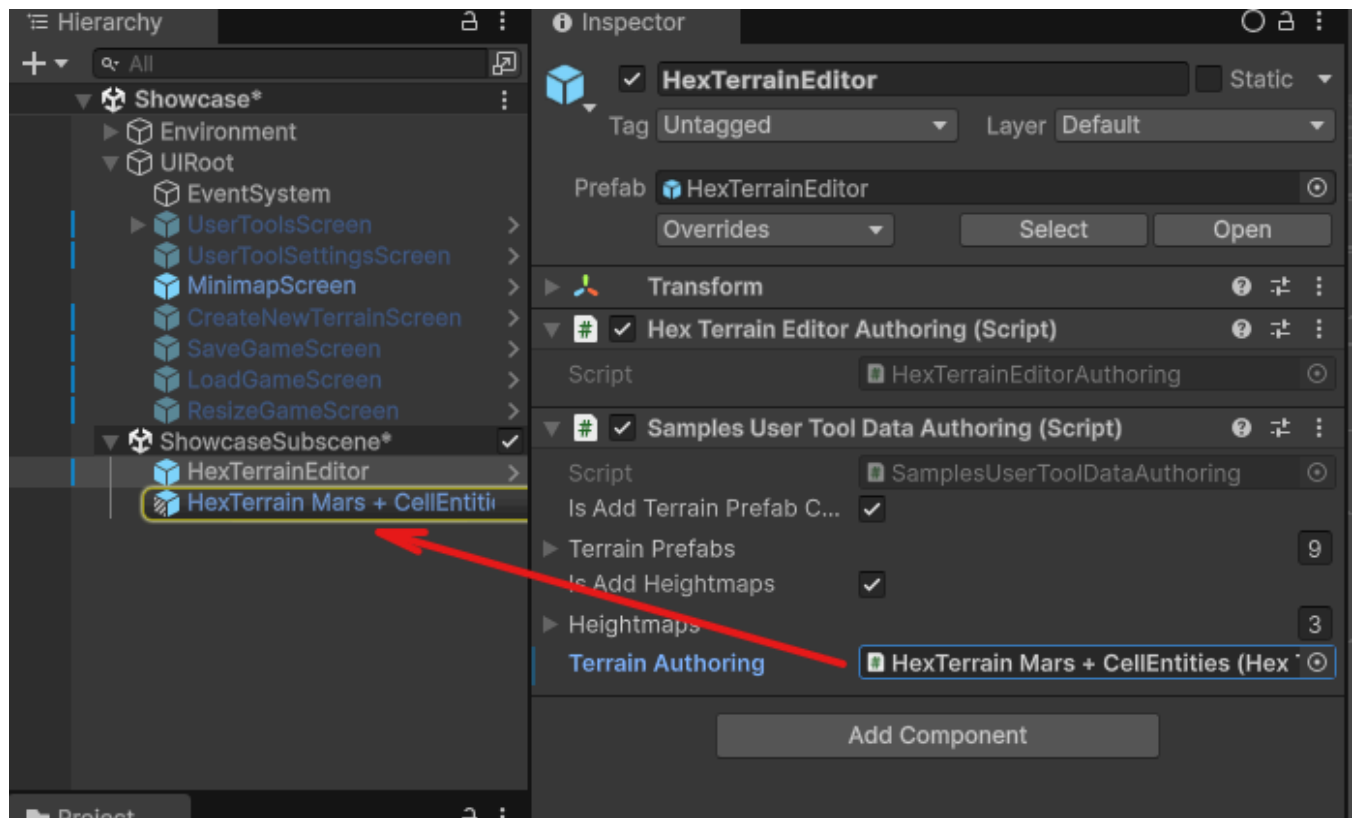
Create terrain on scene load

If you want to have a terrain loaded at scene load, just put on the subscene a terrain prefab. If you have a HexTerrainEditor prefab on the scene, set a reference to the terrain prefab in the `SamplesUserToolDataAuthoring` script in the field `Terrain Authoring` :

1. You can find Terrain prefabs in the Samples folder by path
 "Assets/Samples/Fwt.HexTerrains/1.0.0/HexTerrains_URP/Prefabs" for URP projects and by path
 "Assets/Samples/Fwt.HexTerrains/1.0.0/HexTerrains_HDRP/Prefabs" for HDRP projects



- Put a prefab you want on the "Showcase Subscene" subscene besides the HexTerrainEditor prefab. Then add the reference to this Terrain on the HexTerrainEditor game object at "Terrain Authoring" field of SamplesUserToolDataAuthoring component



Note: if you don't set a reference to the terrain on the HexTerrainEditor, you will not be able to edit a terrain using the User Tools Panel at the bottom center of the screen.

Note: HexTerrainEditor prefab is optional. It is needed for having the User Tools Panel to be created. For your gameplay it might not be needed, so feel free to get rid of it.

How to interact with a Terrain

1. To see how it's done, take a look at interface `ITHexTerrainAPI` and it's example implementation `HexTerrainAPI` class:

```

/// <summary>
/// Example implementation of the IHexTerrainAPI interface.
/// Used to be injected into places outside the ECS World.
/// Contains examples of how to read and write data from the TerrainEntity.
/// Abstracts user code from the TerrainData storage (usually the Entity World).
/// Provides an API for working with a terrain.
/// Usually uses in a UserTools (brushes, etc.).
/// </summary>
public interface IHexTerrainAPI
{
    /// <summary>
    /// Get the TerrainEntity this API is working with.
    /// </summary>
    /// <returns>terrain terrainEntity this API works with</returns>
    Entity GetTerrainEntity();

    /// <summary>
    /// Returns a Terrain Settings if found.
    /// </summary>
    /// <returns>terrain settings if found or null if not found</returns>
    HexTerrainSettings? GetTerrainSettings();

    /// <summary>
    /// returns a data layer from the TerrainEntity <see cref="GetTerrainEntity()"/>
    /// </summary>
    /// <typeparam name="TDataLayer"></typeparam>
    /// <returns>data layer if found, otherwise - null</returns>
    TDataLayer GetDataLayer<TDataLayer>() where TDataLayer : class;

    /// <summary>
    /// Checks if the data layer is supported by the terrain.
    /// Usually means if terrain has this data layer.
    /// </summary>
    /// <typeparam name="TDataLayer">type of the TerrainLayer to check</typeparam>
    /// <returns>true if supports, otherwise - false</returns>
    bool IsLayerSupported<TDataLayer>() where TDataLayer : class;

    /// <summary>
    /// Returns the height map value (byte) of the cell at the specified index.
    /// </summary>
    /// <typeparam name="TSurfaceData">type of surface data to get height from</typeparam>
    /// <param name="cellIndex">cell index to get a height from</param>
    /// <returns>heightmap value if found or null if not found</returns>
    byte? GetCellHeight<TSurfaceData>(int cellIndex) where TSurfaceData : HexSurfaceLayer;

    /// <summary>
    /// Sets the height map value (byte) of the cell at the specified index.
    /// </summary>
    /// <typeparam name="TSurfaceData">type of surface data to set height for</typeparam>
    /// <param name="cellIndex">cell index to set a height for</param>
    /// <param name="height"></param>
    /// <returns>true if value has been set, otherwise - false</returns>

```

```
bool SetCellHeight<TSurfaceData>(int cellIndex, byte height) where TSurfaceData : HexSurfaceLayer;
```

```
/// <summary>
/// Returns the biomes map value (byte) of the cell at the specified index.
/// </summary>
/// <typeparam name="TSurfaceData">type of surface data to get biome from</typeparam>
/// <param name="cellIndex">cell index to get a biome from</param>
/// <returns>biome value if found or null if not found</returns>
byte? GetCellBiome<TSurfaceData>(int cellIndex) where TSurfaceData : HexSurfaceLayer;
```

```
/// <summary>
/// Sets the biomes map value (byte) of the cell at the specified index.
/// </summary>
/// <typeparam name="TSurfaceData">type of surface data to set biome for</typeparam>
/// <param name="cellIndex">cell index to set a biome for</param>
/// <param name="biome">biome value to set</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetCellBiome<TSurfaceData>(int cellIndex, byte biome) where TSurfaceData : HexSurfaceLayer;
```

```
/// <summary>
/// Returns the CellItem from terrain cell at the specified index.
/// </summary>
/// <typeparam name="TItemsLayer">type of cell items layer to get data from</typeparam>
/// <param name="cellIndex">cell index to get a CellItem from</param>
/// <returns>CellItem value if found, otherwise null</returns>
CellItem? GetCellItem<TItemsLayer>(int cellIndex) where TItemsLayer : CellItemsLayer;
```

```
/// <summary>
/// Sets the CellItem from terrain cell at the specified index.
/// </summary>
/// <typeparam name="TItemsLayer">type of cell items layer to set value on</typeparam>
/// <param name="cellIndex">cell index to set CellItem on</param>
/// <param name="item">CellItem value to set on provided cellIndex on provided ItemsData layer</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetCellItem<TItemsLayer>(int cellIndex, CellItem item) where TItemsLayer : CellItemsLayer;
```

```
/// <summary>
/// Returns the CellEntity from terrain cell at the specified index.
/// </summary>
/// <typeparam name="TEntitiesData">type of cell terrainEntity layer to get data from</typeparam>
/// <param name="cellIndex">cell index to get a CellEntity from</param>
/// <returns>CellEntity value if found, otherwise null</returns>
CellEntity? GetCellEntity<TCellEntityLayer>(int cellIndex) where TCellEntityLayer : CellEntitiesLayer;
```

```
/// <summary>
/// Sets the CellEntity from terrain cell at the specified index.
/// </summary>
```

```

    /// <typeparam name="TEntitiesData">type of cell terrainEntity layer to set value on</type
param>
    /// <param name="cellIndex">cell index to set CellEntity on</param>
    /// <param name="item">CellItem value to set on provided cellIndex on provided EntitiesDat
a layer</param>
    /// <returns>true if value has been set, otherwise - false</returns>
    bool SetCellEntity<TCellEntityLayer>(int cellIndex, CellEntity entity) where TCellEntityLa
yer : CellEntitiesLayer;

    /// <summary>
    /// Returns the CellArea from terrain cell at the specified index.
    /// </summary>
    /// <typeparam name="TAreasLayer">type of areas data layer to get value from</typeparam>
    /// <param name="cellIndex">cell index to get value from</param>
    /// <returns>CellArea if found or null if not found</returns>
    byte? GetCellArea<TAreasLayer>(int cellIndex) where TAreasLayer : HexByteAreasLayer;

    /// <summary>
    /// Sets the CellArea from terrain cell at the specified index.
    /// </summary>
    /// <typeparam name="TAreasLayer">type of areas data layer to set value on</typeparam>
    /// <param name="cellIndex">cell index to set value on</param>
    /// <param name="area">area value to set</param>
    /// <returns>true if value has been set, otherwise - false</returns>
    bool SetCellArea<TAreasLayer>(int cellIndex, byte area) where TAreasLayer : HexByteAreasLa
yer;

    /// <summary>
    /// Returns the Raycast data from the TerrainEntity <see cref="HexTerrainRaycastData"/>.
    /// this data contains information about the raycast from mouse to terrain result.
    /// </summary>
    /// <returns>raycast data from the TerrainEntity</returns>
    HexTerrainRaycastData? GetRaycastData();

    /// <summary>
    /// Returns the cell index under the cursor.
    /// </summary>
    /// <returns>cell index if found and raycast hit the terrain, otherwise - false</returns>
    int? GetCellIndexUnderCursor();

    /// <summary>
    /// Returns the cell coordinate under the cursor.
    /// </summary>
    /// <returns>cell coordinate if found and raycast hit the terrain, otherwise - false</retu
rns>
    int2? GetCellCoordUnderCursor();

    /// <summary>
    /// Returns a list of available HexTerrainPrefabConfigs from the UserToolEntity (if any)
    /// </summary>
    /// <typeparam name="THexTerrainPrefabConfig">type of HexTerrainPrefabConfig you have</typ
eparam>
    /// <returns></returns>

```

```

DynamicBuffer<THexTerrainPrefabConfig> GetHexTerrainPrefabConfigsBuffer<THexTerrainPrefabC
onfig>()
    where THexTerrainPrefabConfig : unmanaged, IBufferElementData, IHexTerrainPrefabConfi
g;

/// <summary>
/// Returns a list of available HexTerrainPrefabConfigs from the UserToolEntity (if any).
/// This list can be sent to the UI to display available terrain prefabs for the user to s
elect from.
/// </summary>
/// <returns>collection of hex terrain prefab configs for user to select from</returns>
IList<IHexTerrainPrefabConfig> GetHexTerrainPrefabConfigsList();

/// <summary>
/// Returns the brush view from the TerrainEntity <see cref="HexTerrainBrushView"/>.
/// </summary>
/// <returns>brush view from the TerrainEntity if found, otherwise - null</returns>
HexTerrainBrushView GetBrushView();

/// <summary>
/// Returns the brush size from the TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <returns>brush size if found, otherwise - null</returns>
int? GetBrushSize();

/// <summary>
/// Sets the brush size on the TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <param name="brushSize">new brush size</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetBrushSize(int brushSize);

/// <summary>
/// Returns true if the brush on TerrainEntity is resizable.
/// </summary>
/// <returns>is brush found and resizable</returns>
bool GetIsResizableBrush();

/// <summary>
/// Sets the IsResizable value of the brush on TerrainEntity <see cref="HexTerrainBrush"/
>.
/// </summary>
/// <param name="isResizableBrush">new value for IsResizable</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetIsResizableBrush(bool isResizableBrush);

/// <summary>
/// Returns the brush opacity from the TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <returns>brush opacity if brush is found, otherwise - null</returns>
float? GetBrushOpacity();

```

```

/// <summary>
/// Sets the brush opacity on the TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <param name="opacity">new opacity value (0..1)</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetBrushOpacity(float opacity);

/// <summary>
/// Returns true if the brush on TerrainEntity is visible.
/// </summary>
/// <returns>true if brush is found and is visible, otherwise - false</returns>
bool GetIsVisibleBrush();

/// <summary>
/// Sets the IsVisible value of the brush on TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <param name="isVisibleBrush">new IsVisible value</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetIsVisibleBrush(bool isVisibleBrush);

/// <summary>
/// Returns the brush color from the TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <returns>brush color if found, otherwise - false</returns>
Color32? GetBrushColor();

/// <summary>
/// Sets the brush color on the TerrainEntity <see cref="HexTerrainBrush"/>.
/// </summary>
/// <param name="color">new brush color</param>
/// <returns>true if value has been set, otherwise - false</returns>
bool SetBrushColor(Color32 color);

/// <summary>
/// Returns the view mode of the TerrainEntity <see cref="HexTerrainViewMode"/>.
/// </summary>
/// <returns>view mode if found, otherwise - false</returns>
byte GetViewMode();

/// <summary>
/// Returns true if terrain is visible, false is invisible, null if no terrain with visibility found<see cref="HexTerrainVisibility"/>.
/// </summary>
/// <returns>true if terrain is visible, false is invisible, null if no terrain with visibility found</returns>
bool? GetIsTerrainVisible();

/// <summary>
/// Sets terrain visibility. <see cref="HexTerrainVisibility"/>
/// Returns true if value was set
/// </summary>
/// <param name="isVisible">New visibility value for terrain</param>
/// <returns>True if value was set, otherwise - false</returns>

```

```

    bool SetIsTerrainVisible(bool isVisible);

    /// <summary>
    /// Toggles the visibility of the terrain on the TerrainEntity <see cref="HexTerrainVisibi
lity"/>.
    /// </summary>
    /// <returns>True if terrain now visible, false is invisible, null if no terrain with visi
bility found</returns>
    bool? ToggleIsTerrainVisible();

    /// <summary>
    /// Sets the view mode on the TerrainEntity <see cref="HexTerrainViewMode"/>.
    /// </summary>
    /// <param name="viewMode">new view mode</param>
    /// <returns>true if value has been set, otherwise - false</returns>
    bool SetViewMode(byte viewMode);

    /// <summary>
    /// Saves the TerrainEntity to a file.
    /// </summary>
    /// <param name="filePath">file path to save a terrain into</param>
    /// <returns>true if terrain has been saved, otherwise - false</returns>
    bool SaveTerrain(string filePath);

    /// <summary>
    /// Loads the TerrainEntity from a file.
    /// </summary>
    /// <param name="filePath">file path to load a terrain from</param>
    /// <returns>true if loaded, otherwise - false</returns>
    bool LoadTerrain(string filePath);

    /// <summary>
    /// Resizes the TerrainEntity <see cref="HexTerrainSettings.TerrainSize"/>.
    /// </summary>
    /// <param name="terrainSize">new terrain size</param>
    /// <returns>true if was resized, otherwise - false</returns>
    bool ResizeTerrain(int2 terrainSize);

    /// <summary>
    /// Destroys a terrain entity provided.
    /// </summary>
    /// <param name="terrainEntity">terrain entity to destroy</param>
    /// <returns>true if was destroyed, otherwise - false</returns>
    bool DestroyTerrain(Entity terrainEntity);

    /// <summary>
    /// Creates a new terrain terrainEntity by instantiating provided prefab.
    /// </summary>
    /// <param name="terrainPrefab">prefab of the terrainEntity to create</param>
    /// <returns>instance of the created terrainEntity</returns>
    Entity CreateNewTerrainEntity(Entity terrainPrefab);

    /// <summary>

```

```

    /// Creates a new terrain entity with the specified settings.
    /// Destroys existing terrain
    /// </summary>
    /// <param name="terrainPrefabConfig">prefab config of the terrainEntity to create</param>
    /// <param name="terrainSettings">terrain settings to overwrite</param>
    /// <returns>instance of the created terrainEntity</returns>
    Entity CreateNewTerrain<TPrefabConfig>(TPrefabConfig terrainPrefabConfig, HexTerrainSettings terrainSettings) where TPrefabConfig : IHexTerrainPrefabConfig;

    /// <summary>
    /// Creates a new screen of the specified type.
    /// </summary>
    /// <typeparam name="TScreen">type of the screen to create</typeparam>
    /// <returns>screen instance</returns>
    TScreen CreateUIScreen<TScreen>() where TScreen : UIScreen;

    /// <summary>
    /// Gets an existing screen of the specified type, or creates a new one if it doesn't exist.
    /// </summary>
    /// <typeparam name="TScreen">type of the screen to get or create</typeparam>
    /// <returns>screen instance</returns>
    TScreen GetOrCreateUIScreen<TScreen>() where TScreen : UIScreen;

    /// <summary>
    /// Gets an existing screen of the specified type if found.
    /// </summary>
    /// <typeparam name="TScreen">type of the screen to get</typeparam>
    /// <returns></returns>
    TScreen GetUIScreen<TScreen>() where TScreen : UIScreen;

    /// <summary>
    /// Destroys the specified screen.
    /// </summary>
    /// <typeparam name="TScreen">type of a screen to destroy</typeparam>
    /// <returns>true if the screen was destroyed successfully; otherwise, false</returns>
    bool DestroyUIScreen<TScreen>() where TScreen : UIScreen;

    /// <summary>
    /// Destroys the specified screen instance.
    /// </summary>
    /// <param name="screen">screen instance to destroy</param>
    /// <returns>true if the screen was destroyed successfully; otherwise, false</returns>
    bool DestroyUIScreen(UIScreen screen);
}

```

2. In general all the interactions are done by accessing components on the Terrain Entity:

```

/// <summary>
/// returns a data layer from the TerrainEntity <see cref="TerrainEntity"/>
/// </summary>
/// <typeparam name="TDataLayer"></typeparam>
/// <returns></returns>
public virtual TDataLayer GetDataLayer<TDataLayer>() where TDataLayer : class
{
    if (!EntityManager.Exists(TerrainEntity) || !EntityManager.TryGetComponentObject<TDataLayer>(TerrainEntity, out var dataLayer))
    {
        return default;
    }
    return dataLayer;
}

/// <summary>
/// Returns the height map value (byte) of the cell at the specified index.
/// </summary>
/// <typeparam name="TSurfaceData">type of surface data to get height from</typeparam>
/// <param name="cellIndex">cell index to get a height from</param>
/// <returns>heightmap value if found or null if not found</returns>
public virtual byte? GetCellHeight<TSurfaceData>(int cellIndex) where TSurfaceData : HexSurfaceLayer
{
    var surfaceData = GetDataLayer<TSurfaceData>();
    if (surfaceData == null)
    {
        return null;
    }

    return surfaceData.GetCellHeight(cellIndex);
}

/// <summary>
/// Sets the height map value (byte) of the cell at the specified index.
/// </summary>
/// <typeparam name="TSurfaceData">type of surface data to set height for</typeparam>
/// <param name="cellIndex">cell index to set a height for</param>
/// <param name="height"></param>
/// <returns>true if value has been set, otherwise - false</returns>
public virtual bool SetCellHeight<TSurfaceData>(int cellIndex, byte height) where TSurfaceData : HexSurfaceLayer
{
    var surfaceData = GetDataLayer<TSurfaceData>();
    if (surfaceData == null)
    {
        return false;
    }
    return surfaceData.SetCellHeight(cellIndex, height);
}

```

3. Terrain Entity contains a set of Terrain Layers (see Terrain Layers (terrain-layers.html))

- Surface Layers (Ground, Water, add your own) Contains Heightmap and Biomesmap.

```
public abstract partial class HexSurfaceLayer : IDisposable
{
    ...

    /// <summary>
    /// HeightMap data layer. Contains a byte height value per cell <see cref="HeightMap
    DataLayer"/>
    /// </summary>
    public HeightMapDataLayer HeightMap { get; protected set; }

    /// <summary>
    /// BiomesMap data layer. Contains biome index per cell <see cref="CellBiomesDataLay
    er"/>
    /// </summary>
    public CellBiomesDataLayer BiomesMap { get; protected set; }

    ...
}
```

- Surface Render Layer (HexSurfaceRenderLayer<TSurfaceLayer>) Contains generated meshes for the respective Surface Layer.

```
public abstract partial class HexSurfaceRenderLayer : IDisposable
{
    ...

    /// <summary>
    /// Data layer that stores meshes for the hex terrain chunks.
    /// </summary>
    public ChunkMeshesDataLayer ChunkMeshes;

    ...
}
```

- ByteArea Layers (Countries, Provinces, add your own) Contains a byte map (byte per terrain cell).

```

public abstract partial class ByteAreasLayer : IDisposable
{
    ...

    /// <summary>
    /// Area index per cell
    /// </summary>
    public CellByteAreaDataLayer CellAreaMap { get; set; }

    ...
}

```

- Cell Items Layers (Cell Items, add your own) Contains a CellItems map (Cell Item per terrain cell). Cell Item is a static visual object that is painted on the cell (up to 8 submeshes per CellItem).

```

public abstract class CellItemsLayer : IDisposable
{
    ...

    /// <summary>
    /// Data layer for storing a CellItem per cell
    /// </summary>
    public CellItemDataLayer CellItems;

    ...
}

```

- Cell Entities Layers (Cell Entities, add your own) Contains a CellEntities map (Cell Entity per terrain cell) Cell Entity is a full-fledged Entity with any components on it.

```

public abstract class CellEntitiesLayer : IDisposable
{
    ...

    /// <summary>
    /// Data layer for storing a CellEntity per cell
    /// </summary>
    public HexCellEntityDataLayer CellEntities;

    ...
}

```

4. As you can see, every Terrain Layer contains one or more Data Layers. Data Layer is a container for (usually) collection of data like byte, CellItem, Color32, etc. See more about DataLayers ([data-layers.html](#)). The main point of the DataLayer is that it allows to track Read and Write job handlers, so you can schedule jobs to access or modify a DataLayer's data safely.



Terrain Layers Overview

What is a Terrain Layer?

In the Hex Terrains Framework, each terrain entity consists of several **Terrain Layers**, each responsible for managing a specific kind of spatial data or functionality over the hex grid. These layers are modular, extensible, and powered by job-safe `DataLayer` components.

All layers share a common pattern:

- Store core data per-cell or per-chunk using `DataLayer`s.
- Optionally include color maps for visualization and minimaps.
- Support reinitialization, cleanup, and job completion workflows.



Terrain Layer Types

There are four primary layer types included in the framework:

Layer Type	Purpose	Example Class
<code>SurfaceLayer</code>	Defines core physical terrain (height, biome, surface mesh)	<code>HexGroundLayer</code>
<code>AreasLayer</code>	Stores area/country/province indices per cell	<code>HexCountriesLayer</code>
<code>CellItemsLayer</code>	Stores static objects per cell, renderable with instancing	<code>CellItemPropsLayer</code>
<code>CellEntitiesLayer</code>	Stores a full ECS entity per cell	<code>CellEntityPropsLayer</code>



Surface Layer

Represents visible terrain features like land or water.

Key Features:

- Stores `HeightMapDataLayer` and `CellBiomesDataLayer`
- Contains a `CellColorsDataLayer` for visual overlays
- Linked to a `HexSurfaceRenderLayer<T>` for rendering

Example: HexGroundLayer

This is a simple implementation of `HexSurfaceLayer` for ground terrain.

```
[ChunkSerializable]  
public partial class HexGroundLayer : HexSurfaceLayer, IComponentData { }
```

Render Data

Paired with `HexGroundRenderLayer`, which inherits `HexSurfaceRenderLayer<HexGroundLayer>` and handles:

- Mesh generation from data layers
 - Material configs per view mode
 - Dirty flag tracking and cleanup
-



Areas Layer

Used to mark cells with region identifiers, such as provinces or countries.

Example: HexCountriesLayer

```
public partial class HexCountriesLayer : HexByteAreasLayer, IComponentData { }
```

Internally uses:

- `NativeListChunkedDataLayer<byte>` for area indices
 - `CellColorsDataLayer` for coloring each area
-



Cell Items Layer

Stores static renderable objects per cell using GPU instancing (`Graphics.DrawMeshInstanced`).

Key Components:

- `CellItemDataLayer` – index into a shared list of item types
- `CellTransformDataLayer` – transformation matrix per cell
- `ItemsColorMap` – overlay color per item
- `RenderEntitiesConfig` – defines instancing behavior
- `RenderEntities` – stores active render entity instances

Example: CellItemPropsLayer

```
public class CellItemPropsLayer : CellItemsLayer, IComponentData { }
```

Utility Methods:

- `SetCellItem(cellIndex, item)`

- `SetAllCellsItem(item)`
- `Init(settings, copyItemConfigsFrom, renderConfig)`
- `Resize(newSize)`
- `Dispose()`

Render Pipeline: Cell items are rendered based on their index and corresponding `RenderEntitySettings` .



Cell Entities Layer

Stores an entire ECS entity per cell, useful for dynamic agents, buildings, or gameplay units.

Example: `CellEntityPropsLayer`

```
[ChunkSerializable]
public class CellEntityPropsLayer : CellEntitiesLayer, IComponentData, IDisposable { }
```

Each cell can store:

- A reference to a full entity
- A color (via `CellColorsDataLayer`)
- A transform if needed

Like other layers, it implements:

- `Init()`
 - `SetAllDirty(bool)`
 - `Cleanup()` , `CleanupAsync()`
 - `CompleteAllJobs()`
-



Color Maps

Every layer includes a `CellColorsDataLayer` or `ItemsColorMap` , which stores per-cell `Color32` values to:

- Support terrain visual overlays
- Fill GPU textures used in terrain materials or minimaps
- Provide diagnostic feedback in the editor

To apply the color map:

```
public void ApplyColorsToTexture(Texture2D texture, CellColorsDataLayer colorsDataLayer)
{
    colorsDataLayer.ApplyColorsToTexture(Texture2D texture);
}
```

Lifecycle Hooks

Each TerrainLayer supports:

- **Initialization** (Init)
- **Resizing** (Resize)
- **Dirty flag control** (SetAllDirty(bool))
- **Cleanup** (Cleanup , CleanupAsync)
- **Disposal** (Dispose)
- **Job sync** (CompleteAllJobs)

These ensure safe parallel updates and smooth interoperability with Unity's ECS and Jobs system.

Summary

Feature	Surface	Areas	Items	Entities
Height & Biome	✓	✗	✗	✗
Area Index (byte)	✗	✓	✗	✗
Static Rendered Objects	✗	✗	✓	✗
Dynamic ECS Entities	✗	✗	✗	✓
Cell Color Map	✓	✓	✓	✓
Render Mesh Support	✓	✗	✓	✗

Hex Terrains Framework – Data Layer System

Overview

The **Hex Terrains Framework** features a robust, job-safe data layer system for managing large-scale hexagonal terrain data using Unity DOTS. Each **terrain** is an entity composed of one or more **TerrainLayers**, which encapsulate specific data (e.g., surface, items, areas) in a structured, extensible format. At the core of each **TerrainLayer** is a **DataLayer**, a highly-performant container with built-in support for job dependencies, chunk-based dirty tracking, and safe disposal.

What is a DataLayer?

A **DataLayer** is the foundational building block for storing and managing layer-specific data across your terrain. It:

- Stores data in managed or unmanaged structures
- Tracks read/write `JobHandle` dependencies for safety and performance
- Supports immediate and deferred read/write access
- Tracks dirty states and change versions for efficient updates
- Can be extended for arrays, lists, maps, or custom structures

Each **DataLayer** is either:

- **Flat** – inherited from `DataLayer`, tracks global state (e.g., `ArrayDataLayer`, `ListDataLayer`)
 - **Chunked** – inherited from `ChunkedDataLayer`, tracks dirty state on a per-chunk basis (e.g., `ArrayChunkedDataLayer`, `NativeListChunkedDataLayer`)
-

DataLayer Types

Flat Data Layers (`DataLayer`)

These layers manage data in simple structures such as arrays or lists, without spatial chunk tracking:

Class Name	Description
<code>ArrayDataLayer<T></code>	Backed by a managed array
<code>ListDataLayer<T></code>	Backed by a managed list

Class Name	Description
NativeArrayDataLayer<T>	Uses <code>NativeArray<T></code>
NativeListDataLayer<T>	Uses <code>NativeList<T></code>
DictionaryDataLayer<TKey, TValue>	Uses managed dictionary
NativeHashMapDataLayer<TKey, T>	Uses <code>NativeHashMap</code>
NativeParallelHashMapDataLayer	Uses <code>NativeParallelHashMap</code>
...	More variants for hash sets and multi-maps

All these layers implement a common pattern:

- `Init()`, `GetData()`, `SetData()`
- `TryGetData()`, `TrySetData()`
- `DisposeAllItems()`, `DisposeExcessItems()`

Chunked Data Layers (`ChunkedDataLayer`)

These layers extend `DataLayer` with **dirty chunk tracking**, useful for marking and updating only modified portions of a terrain.

Class Name	Description
ArrayChunkedDataLayer<T>	Managed array with chunk tracking
ListChunkedDataLayer<T>	Managed list with chunk tracking
NativeArrayChunkedDataLayer<T>	Unmanaged array (<code>NativeArray</code>)
NativeListChunkedDataLayer<T>	Unmanaged list (<code>NativeList</code>)

Chunked layers track:

- `ChunkDirtyGrid` : flags for which chunks are dirty
- `DirtyChunks` : hash set of indices that need update
- `ChunkSize`, `ChunkGridSize` : defines how data is divided spatially

They also provide methods like:

- `SetCellDirty(cellIndex, checkEdges)`
- `SetAllChunksDirty()`, `ClearAllChunksDirty()`
- `MergeChunkDirtyGrids(...)` – for combining dirty flags from multiple layers



Job Safety and Dependency Management

All layers support Unity's `JobHandle` dependency tracking to ensure safety when scheduling or accessing data.

Usage Patterns

Operation	Method	Notes
Immediate Read	<code>OpenToRead()</code>	Completes write jobs
Immediate Write	<code>OpenToWrite()</code>	Completes read and write jobs
Schedule Read Job	<code>PrepareToRead()</code>	Use returned <code>JobHandle</code> as dependency
Schedule Write Job	<code>PrepareToWrite()</code>	Use returned <code>JobHandle</code> as dependency
Register Job Completion	<code>AddRead/WriteDependency(JobHandle)</code>	Used in systems

These mechanisms allow multiple systems and jobs to work with terrain layers without risking race conditions or data corruption.



Disposal and Cleanup

If a `DataLayer` holds items that implement `IDisposable`, it will automatically call `.Dispose()` when resized or disposed.

- `IsDisposableItems` : flag to enable per-item disposal
 - `DisposeExcessItems(int newSize)` : disposes unused items when shrinking
 - `DisposeAllItems()` : disposes everything
 - `Dispose()` : cleans up all resources and dependencies
-



Extending DataLayers

To add your own custom layer type:

1. Inherit from `DataLayer` or `ChunkedDataLayer`
 2. Implement required properties like `Length`, and override `Init`, `GetData`, `SetData`
 3. Use appropriate container (`List<T>`, `NativeArray<T>`, etc.)
 4. Optionally implement chunk tracking if spatial change detection is needed
-



Camera Controller System

The Hex Terrains Framework includes a built-in **camera controller system** that allows users to smoothly move and zoom the camera across a hex terrain during gameplay or editing. This system is built on top of Unity ECS and integrates seamlessly with both UI input and ECS logic.

Overview

The camera controller is composed of the following parts:

- **Authoring + Baking:** Declares movement keys and speed settings.
- **ECS Systems:** Create, update, and clean up camera controller components.
- **Scene Component:** A `CameraControllerComponentBase` instance handles zoom behavior and actual camera movement.

This design enables **camera panning**, **zooming**, and **follow behavior** over ECS-controlled hex terrains with minimal boilerplate.

Components

CameraControllerAuthoring (MonoBehaviour)

This Unity MonoBehaviour is placed on a GameObject to configure the camera movement and zoom behavior. It includes:

- Movement input keys (WASD or others)
- Acceleration key (e.g., Shift)
- Speed settings

It gets baked into a `CreateCameraControllerRequest` component.

```
public class CameraControllerAuthoring : MonoBehaviour
{
    public KeyCode MoveUpKey = KeyCode.W;
    public KeyCode MoveDownKey = KeyCode.S;
    public float MovementSpeed = 10f;
    ...
}
```

CreateCameraControllerRequest (IComponentData)

Added to a terrain entity during baking. When processed, it creates a runtime camera controller for the terrain.

```
public struct CreateCameraControllerRequest : IComponentData
{
    public KeyCode MoveUpKey;
    public float MovementSpeed;
    public bool IsFindCameraControllerComponentOnScene;
}
```

CameraControllerTarget (ICleanupComponentData)

Attached to a terrain entity, it contains:

- Current movement input config
- Camera target local position
- Pointer to `CameraControllerComponentBase` `MonoBehaviour`
- `UnityEngine.Transform` for target `GameObject` movement

Systems

CreateCameraControllerSystem

- Executes in the `HexTerrainsInitializationGroup`
- Detects `CreateCameraControllerRequest` and adds `CameraControllerTarget`
- Instantiates a new `Transform` for the camera to follow
- Optionally binds a `CameraControllerComponentBase` found in the scene

Automatically positions the camera over the center of the terrain at start.

UpdateCameraControllerSystemBase

- Handles input (`W`, `A`, `S`, `D`, `Shift`) and applies movement to the camera target
- Wraps around terrain borders if `IsConnectedHorizontally/Vertically` is set
- Updates zoom or world position if any movement occurred
- Calls `CameraControllerComponentBase.UpdatePosition()`
- Updates the minimap view via `IMinimapScreen`

Ignores input if the pointer is over UI (using
`EventSystem.current.IsPointerOverGameObject()`)

DestroyCameraControllerSystem

- Executes in `HexTerrainsCleanupGroup`
 - Cleans up camera targets when terrain becomes invisible or is removed
-

Camera Controller Component

CameraControllerComponentBase (MonoBehaviour)

A Unity-side component for zooming and final camera positioning. You can implement custom behaviors by deriving from this class. For example, it supports:

- `CurrentZoom` value in range `[0-1]`
 - Computed offset between `MinZoomOffset` and `MaxZoomOffset`
 - `SetTarget()` and `UpdatePosition()` abstract methods
-

How to Use

1. Add `CameraControllerAuthoring` to a `GameObject`.
 2. Add a derived implementation of `CameraControllerComponentBase` to the scene.
 3. Assign your terrain entity with the authoring component.
 4. At runtime, camera controller will be initialized and updated automatically.
 5. Optionally override `UpdateCameraControllerSystemBase.GetMinimapScreen()` to sync with your UI.
-

Extending

- You can create custom camera behavior by subclassing `CameraControllerComponentBase`.
 - For advanced use cases (e.g., cinematic camera), you may implement your own systems and disable the default ones.
-

Sample Camera Controller (SamplesCameraControllerComponent)

To help users quickly integrate and test the camera system, the Hex Terrains Framework includes a ready-to-use `MonoBehaviour`: `SamplesCameraControllerComponent`. This is a simple but flexible camera rig with zooming, panning, and edge wrapping.

Purpose

This component demonstrates how to:

- Integrate the ECS-based camera logic with a Unity `Camera` component
- Follow a target (usually a terrain anchor or camera proxy)

- Implement smooth zooming using the mouse scroll wheel
- Avoid UI conflicts (prevents movement if pointer is over UI)
- Enforce terrain boundaries and edge-wrapping logic



Inherits From

```
public class SamplesCameraControllerComponent : CameraControllerComponentBase
```



Features

- **Zoom Control** Uses `Input.mouseScrollDelta.y` to change zoom, clamped between 0 and 1.
 - **Target Following** The camera follows a world `Transform` provided by ECS using `SetTarget()`.
 - **Zoom Modifier Keys** If the user is holding modifier keys like `LeftAlt` or `LeftControl`, zoom is temporarily disabled to avoid accidental input.
 - **Acceleration** Holding the acceleration key (e.g., Shift) increases zoom speed via `AccelerationMultiplier`.
 - **UI-Aware Input** Prevents updates when the user interacts with UI using `EventSystem.current.IsPointerOverGameObject()`.
-



Sample Usage

```
public Camera camera;  
public SamplesCameraControllerComponent controller;  
  
controller.Camera = camera;  
controller.SetTarget(cameraFollowTransform); // Set by ECS at runtime
```



Zoom Calculation Logic

```
var scrollDelta = Input.mouseScrollDelta.y * ZoomSpeed;  
CurrentZoom += scrollDelta * multiplier;  
CurrentZoom = Mathf.Clamp(CurrentZoom, 0, 1);
```

The final zoom offset is applied to the camera using:

```
Camera.transform.position = TargetTransform.TransformPoint(CurrentZoomOffset);  
Camera.transform.LookAt(TargetTransform);
```

Zoom Lock Keys

By default, zoom is disabled while any of the following keys are held:

```
KeyCode.LeftAlt  
KeyCode.LeftControl
```

This is defined in:

```
public List<KeyCode> PreventZoomKeys
```

You can customize this list at runtime or via inspector.

Integration Tip

This is a great starting point for integrating with custom terrain views, minimaps, or input contexts. For a more advanced camera (e.g., orbit or drag-scroll), simply subclass this component or extend `CameraControllerComponentBase`.

Hex Terrains Framework: System Execution Pipeline

The simulation logic of the Hex Terrains Framework is structured into well-defined **execution phases**, organized through a hierarchy of `ComponentSystemGroup`s. This design ensures that all systems run in a deterministic and performant order, respecting job dependencies and data lifecycle phases such as initialization, simulation, rendering, and cleanup.

Each major phase of the pipeline is represented by a dedicated system group and often accompanied by a corresponding **barrier system** (`EntityCommandBufferSystem`) to queue structural changes safely at the end of the phase.

Overview of Execution Order

The system groups are executed in the following sequence throughout each frame:

1. Initialization Phase

- `HexTerrainsCleanupGroup`
- `CreateHexTerrainsGroup`

2. Simulation Phase

- `HexTerrainsInitializationGroup`
- `HexTerrainsStartPreRenderGroup`

3. Late Simulation Phase

- `HexTerrainsFinishPreRenderGroup`

4. Presentation Phase

- `HexTerrainsRenderGroup`

Each group is scheduled inside a broader Unity ECS phase (`InitializationSystemGroup` , `SimulationSystemGroup` , `LateSimulationSystemGroup` , `PresentationSystemGroup`) to ensure correct integration into Unity's default update loop.

1. HexTerrainsCleanupGroup

- **Parent:** `InitializationSystemGroup`
- **Order:** Executes **first** in the frame (`OrderFirst = true`)
- **Purpose:** Resets flags, clears buffers, and prepares terrain data for the new simulation frame.
- **Typical Systems:** Reset dirty flags, mark layers for recomputation.



2. CreateHexTerrainsGroup

- **Parent:** InitializationSystemGroup
 - **Order:** Runs **after** HexTerrainsCleanupGroup
 - **Purpose:** Handles the creation of new terrain entities, initialization of ECS components, and data allocation.
-



3. HexTerrainsInitializationGroup

- **Parent:** SimulationSystemGroup
 - **Purpose:** Core simulation logic runs here. This includes terrain analysis, updates to data layers, generation of mesh data, etc.
 - **Barrier:** HexTerrainsInitializationBarrier (executes last in this group)
 - **Use Case:** Job systems that generate or modify ECS data before rendering.
-



4. HexTerrainsStartPreRenderGroup

- **Parent:** SimulationSystemGroup
 - **Order:** Executes **after** HexTerrainsInitializationGroup
 - **Purpose:** Prepares intermediate render data such as chunk visibility masks or terrain overlays.
-



5. HexTerrainsFinishPreRenderGroup

- **Parent:** LateSimulationSystemGroup
 - **Purpose:** Final cleanup and processing just before rendering.
 - **Typical Use:** Combine buffers, finalize render settings, compress or normalize data.
-



6. HexTerrainsRenderGroup

- **Parent:** PresentationSystemGroup
 - **Purpose:** Renders the terrain using data prepared in previous steps.
 - **Barrier:** HexTerrainsRenderBarrier
 - **Use Case:** Rendering systems using `Graphics.DrawMeshInstanced`, material property block application, GPU data upload.
-



Barrier Systems

Each phase has a corresponding **EntityCommandBufferSystem** that ensures deferred structural changes do not break parallel simulation:

Barrier System	Associated Group	Role
HexTerrainsInitializationBarrier	HexTerrainsInitializationGroup	Applies queued entity changes safely
HexTerrainsRenderBarrier	HexTerrainsRenderGroup	Finalizes render commands

Barrier System	Associated Group	Role
<i>(others exist for each group)</i>	<i>(not yet shown)</i>	Consistent pattern throughout

Summary Diagram (Text)

```

[InitializationSystemGroup]
├─ HexTerrainsCleanupGroup (first)
│   └─ HexTerrainsCleanupBarrier
├─ CreateHexTerrainsGroup (after cleanup)
│   └─ CreateHexTerrainsBarrier

[SimulationSystemGroup]
├─ HexTerrainsInitializationGroup
│   └─ HexTerrainsInitializationBarrier
├─ HexTerrainsStartPreRenderGroup
│   └─ HexTerrainsStartPreRenderBarrier

[LateSimulationSystemGroup]
├─ HexTerrainsFinishPreRenderGroup
│   └─ HexTerrainsInitializationBarrier

[PresentationSystemGroup]
├─ HexTerrainsRenderGroup
│   └─ HexTerrainsRenderBarrier

```

Here is the documentation section for the **Chunks Grid System** in your Hex Terrains Framework:

Chunks Grid System

The Hex Terrains Framework divides the terrain into **chunks** to enable efficient rendering, visibility culling, and simulation scaling. The **Chunks Grid System** manages the lifecycle, visibility state, and dynamic spatial organization of these chunks.

Overview

Each terrain is split into a grid of chunks. This system ensures:

- Only **visible chunks** are rendered or in some cases even updated.
- Chunks can dynamically adjust their **transform offsets** for seamless terrain tiling.
- Simulation and rendering can be constrained to **active areas** based on camera visibility.

Core Concepts

ChunksGridData

A component that stores all spatial data related to chunk organization and visibility. It holds:

- Chunk settings (dimensions, count, etc.)
- Visibility flags per chunk (`ChunkVisibility`)
- Chunk metrics (`ChunkMetrics`)
- Dirty flags for dependency management and selective updating

Added by: `CreateChunksGridDataSystem`

ChunkBounds

Each chunk tracks 9 bounds:

- `Bounds_Center` — actual location
- `Bounds_{Direction}` — adjacent bounds used for edge wrapping

These enable neighborhood queries and visual interpolation across chunk edges.

ChunkVisibility

Stores per-chunk visibility information:

- `IsVisible` — true if inside camera frustum

- `VisualPosition` — where the chunk is currently rendered (may differ from actual world position if terrain wraps)
- `ChunkOffset` — logical offset used for virtual terrain scrolling
- `Transform` — final transform matrix applied to the chunk renderer

Lifecycle Systems

✓ **CreateChunksGridDataSystem**

- Group: `CreateHexTerrainsGroup`
- Automatically adds `ChunksGridData` component to any entity with `CreateChunksGridDataRequest`
- Initializes chunk data using terrain settings

```
chunksGridData.Init(request.Settings);
```

▢ **CalcChunkVisibilitySystem**

- Group: `HexTerrainsInitializationGroup`
- Recalculates visible chunks each frame based on camera frustum
- Stores new visibility states in `ChunkVisibility` per chunk
- Uses a `CalcHexChunkVisibilityJob` to run visibility culling in parallel

```
var job = new CalcHexChunkVisibilityJob {
    // Assign metrics, frustum, visibility maps...
};
job.Schedule(terrainSettings.ChunksCount, 16, combinedDeps);
```

This system is the core of **terrain culling and selective rendering**.

🔪 **CleanupChunksGridDataSystem**

- Group: `HexTerrainsCleanupGroup`
- Calls `CleanupAsync()` on `ChunksGridData` when terrain is visible

Authoring Support

ChunksGridDataAuthoring

- Used in the Unity Editor to define chunks grid metadata
- Extracts terrain settings from a referenced `HexTerrainAuthoring`
- On bake, converts this to a `CreateChunksGridDataRequest` via `ChunksGridDataBaker`

Execution Order Summary

System	Group	Purpose
CreateChunksGridDataSystem	CreateHexTerrainsGroup	Initializes chunks on terrain load
CalcChunkVisibilitySystem	HexTerrainsInitializationGroup	Recomputes which chunks are visible
CleanupChunksGridDataSystem	HexTerrainsCleanupGroup	Cleans up chunk data when no longer needed

Terrain Serialization and Deserialization



HexTerrainSerializer

The `HexTerrainSerializer` is a static utility class that provides serialization and deserialization functionality for `HexTerrain` entities in the framework. It is used to persist terrain state to disk and restore it later. This is especially useful for save/load systems, debugging, world editors, or network transfers.

It supports:

- Versioning for backward compatibility
- Standard and custom component serialization
- Flexible stream or file-based operations



Features

- Serializes terrain settings and selected managed components.
- Supports custom serialization logic via user-provided callbacks.
- Backward-compatible versioning system for future-proofing saved data.
- Works entirely with ECS-friendly components and native containers.



Serializer Structure

```
public static class HexTerrainSerializer
```

This class contains all the logic needed to serialize and deserialize a terrain entity.

```
public static class HexTerrainSerializer
```



Versioning

```
public static int Version = 1;
```

- Indicates the serialization format version.
- Future versions may include additional data layers or structural changes.



Saving Terrain

SaveTerrainToFile

```
public static bool SaveTerrainToFile(string filePath, Entity terrainEntity, EntityManager entityManager, CustomSerializerCallback customSerializer)
```

- Saves the terrain to a specified file.
- Internally calls `Serialize`.



Serialization

There are three main overloads for serialization:

```
Serialize(Stream stream, Entity terrainEntity, EntityManager entityManager, CustomSerializerCallback customSerializer)
```

```
Serialize(BinaryWriter writer, Entity terrainEntity, EntityManager entityManager, CustomSerializerCallback customSerializer)
```

```
WriteVersion1(BinaryWriter writer, Entity terrainEntity, EntityManager entityManager, CustomSerializerCallback customSerializer)
```

- Writes the current version number.
- Writes terrain settings using `WriteTerrainSettings`.
- Enumerates all managed components and serializes recognized types:
 - `HexSurfaceLayer`
 - `CellItemsLayer`
 - `CellEntitiesLayer`
 - `HexByteAreasLayer`
- Supports custom serialization logic by invoking the `CustomSerializerCallback`.

Supported Layer Writers:

- `WriteSurfaceData`
- `WriteCellItemsData`
- `WriteCellEntitiesData`
- `WriteTerrainByteAreas`

Component Type Resolution

```
TypeHash.CalculateStableTypeHash(componentType.GetManagedType())
```

Used to store the component type in a format that's stable and consistent.

Deserialization

LoadTerrainFromFile

```
public static bool LoadTerrainFromFile(string filePath, Entity terrainEntity, EntityManager entityManager, CustomDeserializerCallback customDeserializer)
```

Loads terrain data from a file and applies it to the target terrain entity.

Deserialization Methods

Deserialize(Stream stream, ...)

Deserialize(BinaryReader reader, ...)

ReadVersion1(BinaryReader reader, ...)

- Reads and applies the terrain settings.
 - Matches type hashes to existing component types.
 - Deserializes data into existing component objects using:
 - ReadSurfaceData
 - ReadCellItemsData
 - ReadCellEntitiesData
 - ReadTerrainByteAreas
-

Supported Layers

The following managed components are supported out of the box:

Component Type	Write Method	Read Method
HexSurfaceLayer	WriteSurfaceData	ReadSurfaceData
CellItemsLayer	WriteCellItemsData	ReadCellItemsData
CellEntitiesLayer	WriteCellEntitiesData	ReadCellEntitiesData
HexByteAreasLayer	WriteTerrainByteAreas	ReadTerrainByteAreas



Custom Serialization

CustomSerializerCallback

```
public delegate bool CustomSerializerCallback(BinaryWriter writer, object componentObject);
```

Called during write if the user wants to handle custom logic.

CustomDeserializerCallback

```
public delegate bool CustomDeserializerCallback(BinaryReader reader, Type componentType, HexTerrainSettings terrainSettings);
```

Called during read if the user wants to deserialize custom logic.



Usage Example

```
HexTerrainSerializer.SaveTerrainToFile("Assets/terrain.bin", terrainEntity, entityManager, MyCustomSerialize);  
HexTerrainSerializer.LoadTerrainFromFile("Assets/terrain.bin", terrainEntity, entityManager, MyCustomDeserialize);
```



Binary Format Summary

1. Terrain format version (int)
 2. HexTerrainSettings struct (int2, floats, enums)
 3. Component Type Hash (ulong)
 4. Native list data (length + binary contents)
-



Best Practices

- Always complete all jobs (CompleteAllJobs) before saving to ensure consistency.
- Keep serialization layer-specific logic in their respective data layers.
- Use the custom serializer to support user-defined or experimental components.



Surface Rendering

The **Hex Terrains Framework** supports performant rendering of surface layers (like ground or water) using ECS-compatible systems and highly optimized chunk-based rendering workflows. This section explains how rendering works, what components are involved, and how meshes are dynamically generated and drawn for visible terrain chunks.



Rendering Workflow Overview

Surface rendering is split into three main phases:

1. **Create Render Data** The `CreateSurfaceRenderDataSystem` is responsible for creating a `HexSurfaceRenderLayer` instance tied to a specific `HexSurfaceLayer`. This object holds all the render-related data structures including chunk meshes, mesh sources, and render configuration.
 2. **Calculate Mesh Sources (Verts, UV, tris, etc.)** The `CalcGroundMeshSourcesSystem` system calculates mesh data for visible chunks to be later copied into the chunk meshes (regular Unity Mesh).
 3. **Generate Mesh Data** The `CalcSurfaceMeshesSystem` converts the intermediate `MeshSource` data (generated earlier by other systems like `CalcGroundMeshSourcesSystem`) into actual Unity `Mesh` objects per chunk. Only dirty or changed chunks are reprocessed.
 4. **Render Meshes** The `RenderHexSurfaceSystem` uses Unity's `Graphics.DrawMesh` to efficiently render only the visible chunks for the current camera using instancing and render config materials.
-



CreateSurfaceRenderLayerSystem

```
public class CreateSurfaceRenderLayerSystem<TRequest, TSurfaceLayer, TRenderData, TRenderConfig> : SystemBase
```

- **Purpose:** Initializes the render data structure when a `TRequest` component is present on a terrain entity.
- **Output:** Attaches a `TRenderData` component with:
 - Render settings from the request.
 - Render config asset.
 - Camera reference for rendering.
- **Example Output:** Adds `HexGroundRenderLayer` or `HexWaterRenderLayer`.

✚ Render data includes:

- `ChunkMeshSources` (procedural mesh definitions)
- `ChunkMeshes` (final Mesh instances)
- `RenderConfig` (materials and shadow settings)

【Source: `CreateSurfaceRenderLayerSystem.cs`】



CalcSurfaceMeshesSystem

```
public class CalcSurfaceMeshesSystem<TSurfaceData, TRenderData> : SystemBase
```

- **Purpose:** Builds Unity Mesh objects from procedural MeshSource data for each chunk.
- **Key Features:**
 - Skips work if nothing is dirty.
 - Processes only marked dirty chunks via `GeneratedMeshIndexes`.
 - Fills meshes using `MeshSource.FillMesh()` and stores them in the render layer.
- **Optimization:** Once processed, dirty flags are reset to avoid redundant work.

✂ Example:

```
meshSources[chunkIndex].FillMesh(chunkMesh);
```

【Source: `CalcSurfaceMeshesSystem.cs`】



RenderHexSurfaceSystem

```
public class RenderHexSurfaceSystem<TSurfaceData, TRenderData> : SystemBase
```

- **Purpose:** Draws all visible chunk meshes using `Graphics.DrawMesh`.
- **Features:**
 - Uses `ChunkVisibility` to skip non-visible chunks.
 - Supports view mode switching via `HexTerrainViewMode`.
 - Can draw multi-material meshes using submesh indices.

🔍 Pseudocode:

```
foreach (chunkIndex in visibleChunks)
    if (chunk is visible)
        DrawMesh(chunkMeshes[chunkIndex], transform, material)
```

- **Supports:** Shadows (`IsCastShadows`, `IsReceiveShadows`) and instanced draw calls.

Supporting Components

Component / System	Purpose
HexSurfaceRenderLayer	Holds all mesh/render data for surface
MeshSource	Procedural mesh buffer (verts, tris, uvs)
ChunksGridData.ChunkVisibility	Visibility status and transform for each chunk
HexTerrainViewMode	View mode index for rendering config
HexSurfaceRenderConfig	Materials, shadow settings, etc.

Summary

The surface rendering pipeline in Hex Terrains Framework is designed for **maximum performance** and **visual clarity**, dynamically processing and rendering only the chunks that are visible. The system is highly modular, allowing surfaces like ground and water to be rendered using shared logic, with configuration driven by ECS components and per-layer settings.



Terrain Surfaces

Hex Terrains Framework supports multiple surface types (like ground or water), each implemented as a **HexSurfaceLayer**. These surfaces are responsible for storing key data such as height, biomes, and render information. Surfaces are extensible, fully ECS-compatible, and processed with DOTS for maximum performance.

Surface Types

Each surface type is implemented as a class that inherits from `HexSurfaceLayer`. The primary concrete implementations include:

- **HexGroundLayer** – for terrain ground surface (elevation, terrain biomes).
- **WaterSurfaceLayer** (*not shown here, but supported similarly*).

Each surface has associated systems to handle metric calculation and rendering-related preprocessing.



Structure of a Surface Layer

Each `HexSurfaceLayer` holds multiple data layers, such as:

Data Layer	Purpose
HeightMap	Stores per-cell elevation
BiomesMap	Stores biome index per cell
CellColorMap	Stores visualized colors for each cell
CellMetrics	Stores combined metrics derived from height + biome
BiomeColors	A palette for converting biome index to color

All of these layers are implemented as `DataLayer` or `ChunkedDataLayer` objects and benefit from read/write dependency tracking and chunk-level dirty maps for efficient job scheduling.



Data Generation and Setup

Surfaces can be initialized via **authoring components** in the Unity Editor. For example:

Example: HexGroundLayerAuthoring

```
public class HexGroundLayerAuthoring : HexSurfaceLayerAuthoring {}
```

When baked by `HexGroundDataBaker`, this authoring component:

- Sets up height, biomes, and color palette.
- Adds the `HexGroundColorPalette` buffer to the terrain entity.
- Schedules a `CreateHexGroundLayerRequest`.

```
public struct CreateHexGroundLayerRequest : ICreateHexSurfaceRequest, IComponentData
```

This request is then picked up by systems to construct the ECS-side surface representation.

Simulation and Updates

Color Map Generation: `CalcCellBiomeColorsSystem<T>`

This system:

- Compares dirty flags between biomes and color maps.
- Combines chunk-level dirty grids.
- Schedules a `CalculateBiomesColorMapJob` or `CalculateWaterColorMapJob`.
- Writes biome color to `CellColorMap` using the defined palette.

The system ensures color maps update *only when needed*, and processes data in jobs chunk-by-chunk.

Metrics Calculation: `CalcCellMetricsSystem<T>`

Surface metrics such as slope, average height, or relative biome data are calculated here.

- Depends on `HeightMap` and `BiomesMap`.
- Uses `ChunkDirtyGrid` to identify regions needing recalculation.
- Populates `CellMetrics` (metrics and cell position data).

Specialized implementation:

```
public partial class CalcGroundCellMetricsSystem : CalcCellMetricsSystem<HexGroundLayer>
```



Runtime Behavior

- Systems are designed to work independently per terrain, which allows multi-terrain setups.
 - Surfaces automatically react to changes in their layers (e.g., biome or heightmap changes) via their dirty flags and dependency tracking.
-



Color Palette System

For visual feedback (e.g., biomes displayed in the minimap or by coloring cells on terrain):

- Each surface has a color palette (e.g., `HexGroundColorPalette`).
 - The palette is used during biome color map job execution.
 - Supports easy customization and visualization without needing shader-side logic.
-



Custom Surfaces

To create a custom surface:

1. Derive from `HexSurfaceLayer` .
 2. Create an authoring component and baker.
 3. Implement a data creation request (`ICreateHexSurfaceRequest`).
 4. Add custom systems for metric or render preparation as needed.
-

Here's a structured documentation section for the **Terrain Surfaces** system in your Hex Terrains Framework:

Terrain Surfaces

Hex Terrains Framework supports multiple surface types (e.g., ground, water) using a flexible and extensible layer system. Each surface is implemented as a specialized `HexSurfaceLayer` and may store height data, biome information, color maps, and associated rendering data. This section details how terrain surfaces are constructed, managed, raycasted, and updated.

Overview

A **HexSurface** is a conceptual layer of the terrain, such as the ground or water. Each one is implemented via:

- A derived class from `HexSurfaceLayer`
- A `HeightMapDataLayer` for cell heights (as `byte` , scaled to world meters)
- A `CellBiomesDataLayer` for storing biome types per cell
- A `CellColorsDataLayer` for coloring the surface, updated based on biomes or other logic
- A `HexTerrainRenderData` layer to render the surface mesh

Example surface layers:

- `HexGroundLayer` (ground surface)

- `HexWaterLayer` (custom surface type, if added)
- Each implements `ICreateHexSurfaceDataRequest` or uses associated request structs for instantiation

Layer Creation

Surfaces are initialized by systems such as `CreateHexGroundLayerRequest` and `CreateHexSurfaceRequestArgs`, usually via authoring components like `HexGroundLayerAuthoring` or `HexSurfaceLayerAuthoring`. These components describe what kind of surface to create and associate them with a terrain entity.

Example Setup Flow

```
HexGroundLayerAuthoring -> CreateHexGroundLayerRequest
→ System receives request → Creates HexGroundLayer
→ Initializes DataLayers (HeightMap, Biomes, Colors)
```

Cleanup

Each surface is cleaned every frame (if visible) to clear dirty flags or release temporary resources.

- Base class: `CleanupHexSurfaceLayerSystem<TSurface>`
- Example implementation: `CleanupHexGroundLayerSystem`

```
public class CleanupHexGroundLayerSystem
    : CleanupHexSurfaceLayerSystem<HexGroundLayer>
{
    protected override void OnUpdate() { ... }
}
```

The system checks `HexTerrainVisibility.Value` before calling `surface.CleanupAsync()`.

Surface Color Map Calculation

The biomes map drives the visual appearance of terrain via `CalculateBiomesColorMapJob`. This job:

- Iterates over each cell in parallel
- Skips clean (non-dirty) chunks
- Maps the biome index (byte) to a `Color32` using a palette
- Writes results into `CellColorsDataLayer`

```
if (cellValue < Colors.Length)
    CellColorMap[index] = Colors[cellValue];
else
    CellColorMap[index] = DefaultColor;
```

This allows efficient color map recalculation per chunk.

Raycasting

Two raycasting jobs exist for interaction and tooling:

RaycastHexTerrainJob

Raycasts against a **single** surface's heightmap. It:

- Iterates along the ray by steps
- Converts world coordinates to hex cell index
- Checks if the point is under the surface's height

If so, it writes a `HexTerrainRaycastData` containing:

- `IsHit`
- `HitPosition`
- `CellCoord`, `CellIndex`, `ChunkIndex`, `ChunkCoord`

RaycastHexTerrain2Job

Works similarly, but checks **two surfaces simultaneously** (e.g., ground and water) and picks the one with higher height at each cell.

```
var rawHeight = max(Heightmap1[cellIndex], Heightmap2[cellIndex]);
```

This enables correct behavior for raycasts involving layered terrain.

Terrain Metrics

The `CalcHexCellMetricsJob` and `CalcCellMetricsSystem` compute per-cell properties like slope, height, and elevation, used for physics, rendering, or gameplay logic. These systems run after terrain data changes and update internal terrain state accordingly.

Summary

Hex Terrain Surfaces provide an efficient and extensible way to represent elevation, biomes, and visual data. Each surface is ECS-native and fully integrated into the simulation loop, using chunked data layers, dirty tracking, and job scheduling for high performance. Whether you're rendering a mountain range or simulating ocean waves, `HexSurfaceLayer` is the foundation that supports terrain realism in the framework.

User Interface concepts

UI Binding System in Hex Terrains Framework

The UI in Hex Terrains Framework is built on **Unity UI Toolkit** and uses a **data-driven binding system** that seamlessly synchronizes UI screens with underlying runtime data.

This system ensures UI elements always reflect the current application state, and allows runtime tools (like terrain brushes, minimaps, and tool selectors) to respond to user input with minimal boilerplate.

Core Concepts

Component	Role
UIDataSource	Base class for all bindable data. Tracks changes using a <code>Version</code> .
IViewDataReceiver<T>	Interface for screens to receive and apply data sources.
UIScreen<T>	Base class for all UI screens, supports binding and updating.
ToolSettingsScreen<T>	Generic base for tool-specific UI panels.
MinimapScreenBase	Base for minimap UI, receives view data updates from systems.

Data Binding with UIDataSource

All data exposed to UI inherits from `UIDataSource` . This abstract class handles:

- **Change detection** via a `Version` counter
- **Dirty tracking** using `MarkDirty()`
- **UI invalidation** with `CommitChanges()`

```
public abstract class UIDataSource : ICommitChanges {  
    public long Version;  
  
    public virtual void MarkDirty() => Version++;  
    public virtual void CommitChanges() => MarkDirty();  
}
```

When a value changes, `CommitChanges()` increments the `Version` , signaling UI Toolkit to rebind affected elements.

View Data Lifecycle

1. A screen receives a data object via `SetViewData(T viewData)`
2. The screen assigns the data to its root visual element's `.dataSource`
3. UI Toolkit auto-binds fields based on `[CreateProperty]` attributes

```
public virtual void SetViewData(TSettings viewData) {  
    _viewData = viewData;  
    if (TryGetRootElement(out var rootElement)) {  
        rootElement.dataSource = viewData;  
    }  
}
```

This flow is defined in `ToolSettingsScreen<T>` and reused across all UI panels.

Example: Tool Settings Panel

```
public class UniversalToolSettingsScreen : ToolSettingsScreen<UserToolSettingsDataSource> { }
```

- Binds to `UserToolSettingsDataSource`, which is a `UIDataSource`
 - Automatically reflects settings like brush size, tool parameters, and visibility
 - Changes to settings (e.g., via sliders or dropdowns) update the bound data and propagate to the active user tool
-

Example: Minimap UI

The minimap UI is backed by `MinimapScreenDataSource`, a subclass of `UIDataSource`.

Key properties:

```
[CreateProperty] public List<Texture2D> ColorMapTextures;  
[CreateProperty] public List<DisplayStyle> ColorMapTextureVisibilities;  
[CreateProperty] public Vector2 CameraNormalizedPosition;
```

When `SetColorTextures(...)` or `SetViewMode(...)` is called in `MinimapScreenBase`, it updates the data source and commits changes:

```
ViewData.SetVisibleTextureIndexes(visibleIndexes);  
ViewData.CommitChanges();
```

This triggers an immediate UI refresh showing the correct textures and view modes.



UI Message Routing (Optional)

The `IViewDataReceiver<T>` and base `IViewDataReceiver` interfaces support message routing and generalized UI communication.



Benefits

- Declarative, low-code UI architecture
 - Automatic UI refresh on data changes
 - Clean separation between logic and presentation
 - Highly extensible and testable
-



UI Toolkit Custom Controls and Data Templates

In addition to core screen and data binding infrastructure, the Hex Terrains Framework includes a powerful set of **custom UI Toolkit components** to streamline data-driven UI development.

These components support dynamic layout, list rendering, event handling, and runtime template selection.



UIButton: Unified Button Binding

The `UIButton` component enhances Unity's standard `Button` by supporting multiple binding strategies for click handling:

```
[CreateProperty]
public object OnClick { get; set; }

[CreateProperty]
public string OnClickPath { get; set; }
```

Supported click targets include:

- Action
- UnityEvent
- UIButtonDataSource (with an `OnClick` action)
- Property path bindings (using `OnClickPath`)

This flexibility allows UI to invoke methods on view models or data sources declaratively.



SimpleListView: Static Template Lists

The `SimpleListView` displays a list of items using a single `VisualTreeAsset` template:

```
[CreateProperty]
public VisualTreeAsset ItemTemplate { get; set; }

[CreateProperty]
public IList itemsSource { get; set; }
```

For each item in `itemsSource` :

- A view is instantiated using `ItemTemplate`
- Its `.dataSource` is assigned
- If the view implements `ICommitChanges` , it is refreshed automatically

This is ideal for simple collections with consistent UI layout.

UIListView: Event-Driven Selection

`UIListView` extends Unity's native `ListView` with enhanced selection binding. You can assign:

- `UnityEvent<IEnumerable<object>>`
- `UnityEvent<int>`
- `Action<int>` OR `Action<IEnumerable<object>>`

All are invoked automatically when the user changes selection.

```
public object OnSelectionChange { get; set; }
```

Template Selection System

The framework supports **data-driven UI templates** using:

- `TemplateSelectorConfig` : Base for defining default templates
- `TemplateSelectorConfigByType` : Maps C# types to specific templates
- `DataTemplateConfig` : Stores the mapping between type and `VisualTreeAsset`

This system allows runtime UI to render heterogeneous data models with distinct views based on data type.

```
public override VisualTreeAsset GetTemplateForData<TData>(TData data) {
    return GetTemplateForType(typeof(TData));
}
```

TemplateSelectorElement: Dynamic UI Rendering

`TemplateSelectorElement` dynamically selects and instantiates a view template at runtime based on the data it is bound to.

How it works:

1. **Retrieves a data object** from the UI hierarchy using `GetHierarchicalDataSourceContext`
2. **Selects a template** via a `TemplateSelectorConfig`
3. **Creates and binds** a view using the selected template
4. Automatically **updates** when data changes or when reattached to panel

This mechanism is ideal for modular and polymorphic UIs such as:

- Properties panels for different object types
- Inspector-style interfaces
- Minimap overlays with customizable visuals

Summary

Component	Purpose
UIButton	Data-bound click handling with path and object support
SimpleListView	Fast list rendering using one template
UICollection	Flexible list view with event-driven selection
TemplateSelectorConfig	Maps data types to UI templates
TemplateSelectorElement	Renders data using a selected template at runtime



In-Game Terrain Editor

Overview

The Hex Terrains Framework features a powerful in-game terrain editor designed with a **Photoshop-like workflow** in mind. It supports:

- Brush-based editing with size, opacity, and color control
- Modular tools for deforming terrain, painting areas, placing objects, and more
- A runtime state machine to manage tools
- Full UI Toolkit integration via a shared UI system
- Clean separation between UI code and ECS-based terrain data



User Tool System Architecture

Each editor tool is a **state** within a `UserToolStateMachine`, allowing seamless switching between editing modes.

Key Components

Component	Description
<code>UserToolStateMachine</code>	Runtime state machine managing tool transitions
<code>UserToolStateBase</code>	Base class for all tool states; handles activation, brush setup, view mode
<code>IHexTerrainAPI</code>	Interface injected into tools to access terrain editing APIs
<code>UserToolSettings</code>	Data-driven brush/tool parameters
<code>IUISystemAPI</code>	UI integration layer for editor panels and toolbars



Tool Lifecycle

Each tool derives from `UserToolStateBase` and can optionally:

- Initialize a UI panel (`ShowSettingsScreen`)
- Activate a custom brush with settings
- Switch terrain view modes
- Apply edits using `ApplyBrush(...)`

Example Lifecycle Flow

```
[Tool Selected]
  ↓
PrepareToRun()
  ↓
Set View Mode / Show Brush
  ↓
Tick() - per-frame updates
  ↓
ApplyBrush(...) - on user input
  ↓
PrepareToStop() - restore view mode, hide UI
```



Tool Example: Deform Ground

Class: DeformGroundUserToolState

This tool deforms terrain height based on mouse input, key modifiers, and brush size.

```
public class DeformGroundUserToolState : DeformHeightUserToolState<HexGroundLayer>
{
    public override int UserToolType => (int)UserToolTypes.GroundDeformation;
}
```

Modifier Keys

Key	Effect
LeftAlt	Inverts height delta
LeftShift	Applies turbo strength



Tool Example: Paint Countries

Class: PaintCountriesUserToolState

This tool paints area indices (such as countries) onto cells, optionally constrained by elevation.

```
public class PaintCountriesUserToolState : PaintAreasUserToolState<HexCountriesLayer, HexGroundLayer>
{
    public override int UserToolType => (int)UserToolTypes.CountriesPainting;
}
```

Settings

- **Index** – area index (byte 0–255)
- **Min Cell Height** – skip cells below this elevation
- **Max Cell Height** – skip cells above this elevation



IHexTerrainAPI – Bridging Tools and Terrain

The `IHexTerrainAPI` allows tools to interact with terrain layers outside ECS systems:

```
public interface IHexTerrainAPI
{
    void SetIsVisibleBrush(bool isVisible);
    void SetBrushSize(int size);
    void SetBrushColor(Color32 color);
    void SetIsResizableBrush(bool isResizable);
    int GetViewMode();
    void SetViewMode(byte mode);
    byte GetCellHeight<TSurfaceLayer>(int cellIndex) where TSurfaceLayer : HexSurfaceLayer;
    // ... other helper methods
}
```

This ensures tools are lightweight, testable, and do not manipulate ECS state directly.



Creating a New Tool

To implement a new terrain editing tool:

1. **Inherit** from `UserToolStateBase` or a derived class like `BrushUserToolState`
2. **Implement** `ApplyBrush(...)` with your editing logic
3. **Set** a unique `UserToolType` ID
4. **Register** the tool in your state machine
5. **(Optional)** Provide a UI panel by calling `ShowSettingsScreen`

Template

```
public class MyCustomToolState : BrushUserToolState<SomeLayer>
{
    public override int UserToolType => (int)UserToolTypes.MyTool;

    public MyCustomToolState(IHexTerrainAPI hexTerrainAPI) : base(hexTerrainAPI) { }

    public override bool ApplyBrush(...)
    {
        // Custom logic here
    }
}
```



Brushes & Cursors

Brush visuals and interaction are controlled via `IHexTerrainAPI` :

- Size
- Color
- Opacity
- Visibility
- Resizability

All brush properties are dynamically adjusted based on tool and user input.



Tool Settings UI

Tools can define custom settings through data-driven `UserToolSettingsDataSource` :

```
DataSource.AddValue(new UserToolParameterDataSource<int>
{
    Name = "Min Cell Height",
    MinValue = 0,
    MaxValue = 255,
    Value = 3,
    IsClampValue = true,
    IsVisible = true
});
```

The UI system automatically binds these parameters to in-game panels using a common `UIScreen` and `IViewDataReceiver` .



Cleanup and Memory Safety

All tools implement:

- `Dispose()` for cleanup
- `PrepareToStart()` to prepare tool for work and show UI
- `Tick()` to apply per-frame logic
- `PrepareToStop()` to revert view modes and hide UI and more (see `UserToolStateBase` (<https://www.freewebtime.com/hexterrains/api/Fwt.HexTerrains.TerrainEditors.StateMachines.UserToolStateBase.htm>))

Brush-Based Terrain Editing

The **BrushUserTool system** powers the in-game Photoshop-like editing experience within the Hex Terrains Framework. It allows real-time, cell-level modifications of terrain layers using a dynamic, customizable brush. Each tool is implemented as a state in a user tool state machine and inherits from a shared foundation:

`BrushUserToolState` .

Key Features

- Adjustable **brush size** and **opacity**
- Unified API for diverse operations (e.g., painting countries, applying heightmaps, stamping objects)
- Tool-specific logic via overrideable brush application methods
- Data-driven settings exposed via UI
- Supports undo/redo, smooth user interactions, and multi-step operations

Base Class: `BrushUserToolState`

The core abstract base class:

```
BrushUserToolState<TBrushTarget, TDataSource, TUIScreen>
```

- `TBrushTarget` : The layer type affected (e.g., `HexSurfaceLayer` , `CellEntitiesLayer`)
- `TDataSource` : A class describing editable settings (usually `UserToolSettingsDataSource`)
- `TUIScreen` : The settings panel (usually `UniversalToolSettingsScreen`)

Example Implementations

Tool	Inherits From	Purpose
<code>ApplyHeightmapUserToolState</code>	<code>BrushUserToolState<HexSurfaceLayer, ...></code>	Paints heightmap values onto terrain cells
<code>AutoPaintUserToolState</code>	<code>BrushUserToolState<HexSurfaceLayer, ...></code>	Auto-assigns biomes based on elevation
<code>PaintCountriesUserToolState</code>	<code>PaintAreasUserToolState<HexCountriesLayer, ...></code>	Assigns country indices per cell
<code>LevelTerrainUserToolState</code>	<code>BrushUserToolState<HexSurfaceLayer, ...></code>	Sets cells to a specific elevation

Tool	Inherits From	Purpose
StampUserToolStateBase	BrushUserToolState<...>	Copies cell data from one area to another



Brush Lifecycle

Each frame while a brush is active:

1. **Cursor** location is evaluated (`HexTerrainAPI.GetCellCoordUnderCursor()`).
2. **Affected cells** within brush radius are gathered.
3. `ApplyBrush(...)` is invoked for each cell.
4. Layer-specific logic (e.g., biome assignment, height change, object stamping) is applied.
5. UI settings are updated if necessary (e.g., when reading a value).



User-Defined Parameters

Each brush tool can expose user-facing parameters:

```
DataSource.AddValue(new UserToolParameterDataSource<int> {
    Name = "Elevation",
    Value = 0,
    MinValue = 0,
    MaxValue = 255,
    IsVisible = true,
});
```

These parameters are shown in the editor UI and persisted during the session.



Overriding ApplyBrush

Every brush tool must override `ApplyBrush` :

```
public override bool ApplyBrush(int brushMode, int2 cellCoord, int cellIndex, TBrushTarget target, int mouseButton)
```

For example, in `ApplyHeightmapUserToolState` , the method reads a value from a texture and updates the terrain height:

```
var pixel = Heightmap.GetPixelBilinear(progress.x, progress.y);
var height = (byte)math.clamp(pixel.r * scale * 255 + offset, 0, 255);
brushTarget.SetCellHeight(cellIndex, height);
```



Extending BrushUserTool

To create a custom brush tool:

1. Inherit from `BrushUserToolState<TBrushTarget, TDataSource, TUIScreen>`
2. Override `InitDefaultSettings()` to register UI parameters.
3. Override `ApplyBrush(...)` with logic for applying edits.
4. Register your tool in the state machine (`UserToolStateMachine.SwitchStateByUserToolType(...)`).



HexTerrainAPI: Default Implementation of IHexTerrainAPI

The `HexTerrainAPI` class is the default implementation of the `IHexTerrainAPI` interface, designed to bridge **runtime user tools** (e.g., brush operations) with ECS-based **HexTerrain** data structures.

It provides a standardized way for tool states to interact with terrain data, access brush settings, UI screens, and read/write values from data layers such as heightmaps, biomes, cell entities, and areas — all without requiring direct access to the `EntityManager`.



Purpose

- Acts as a **facade** over the `EntityManager`, abstracting terrain access logic.
- Enables tools (like `BrushUserToolState`) to:
 - Modify terrain data (e.g., deform terrain, paint biomes).
 - Read/write brush settings (size, opacity, visibility).
 - Manage terrain lifecycle (create, destroy, resize, save/load).
 - Interact with UI (via injected `IUISystemAPI`).
- Is **injectable** into tool states as `IHexTerrainAPI`.



Core Components

Property	Description
<code>EntityManager</code>	ECS context for terrain access.
<code>TerrainEntity</code>	Current active terrain entity.
<code>UserToolEntity</code>	Tool controller entity (provides prefab configs, settings).
<code>UISystemAPI</code>	Interface to manage UI screens (create, destroy, retrieve).



Terrain Access & Manipulation

You can access and modify ECS terrain data safely through these functions:

- **Generic Data Layer Access**

```
TDataLayer GetDataLayer<TDataLayer>() where TDataLayer : class;  
bool IsLayerSupported<TDataLayer>() where TDataLayer : class;
```

- **Heightmap / Biome Access**

```
byte? GetCellHeight<TSurface>() / SetCellHeight<TSurface>()  
byte? GetCellBiome<TSurface>() / SetCellBiome<TSurface>()
```

- **Cell Items / Entities**

```
CellItem? GetCellItem<T>() / bool SetCellItem<T>()  
CellEntity? GetCellEntity<T>() / bool SetCellEntity<T>()
```

- **Areas**

```
byte? GetCellArea<T>() / bool SetCellArea<T>()
```

- **Raycasting**

```
HexTerrainRaycastData? GetRaycastData()  
int? GetCellIndexUnderCursor()  
int2? GetCellCoordUnderCursor()
```

Brush Manipulation

HexTerrainAPI provides full access to brush properties:

Property	Methods
Size	GetBrushSize(), SetBrushSize(int)
Opacity	GetBrushOpacity(), SetBrushOpacity(float)
Visibility	GetIsVisibleBrush(), SetIsVisibleBrush(bool)
Resizability	GetIsResizableBrush(), SetIsResizableBrush(bool)
Color	GetBrushColor(), SetBrushColor(Color32)
View Mode	GetViewMode(), SetViewMode(byte)
View	GetBrushView()

Terrain Lifecycle Management

Operation	Method
Save / Load	SaveTerrain(path) , LoadTerrain(path)
Resize	ResizeTerrain(int2)
Destroy	DestroyTerrain(entity)
Create	CreateNewTerrainEntity(prefab) CreateNewTerrain(config, settings)

UI Integration

Operation	Method
Create screen	CreateUIScreen<T>()
Get screen	GetUIScreen<T>()
Get or create	GetOrCreateUIScreen<T>()
Destroy screen	DestroyUIScreen<T>() / DestroyUIScreen(UIScreen)

Extensibility

Developers can:

- Use this class as-is.
- Subclass it to extend behavior.
- Implement `ISHexTerrainAPI` from scratch for alternative logic.

This makes `HexTerrainAPI` both **powerful** and **flexible**, fitting a wide range of in-game editor use cases.
